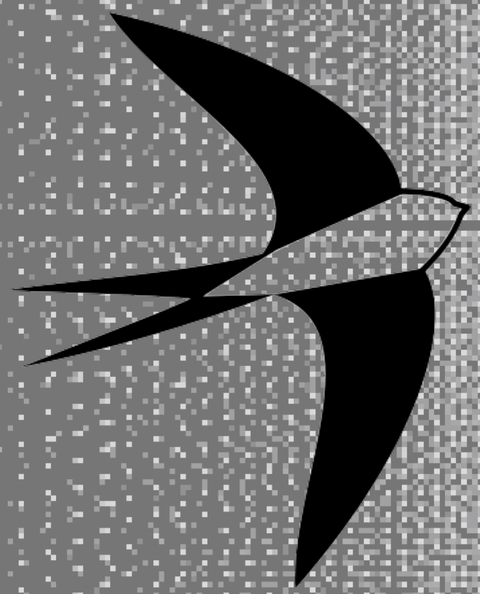
SWIFT-Zoom: Efficient Scalable Zoom Simulations

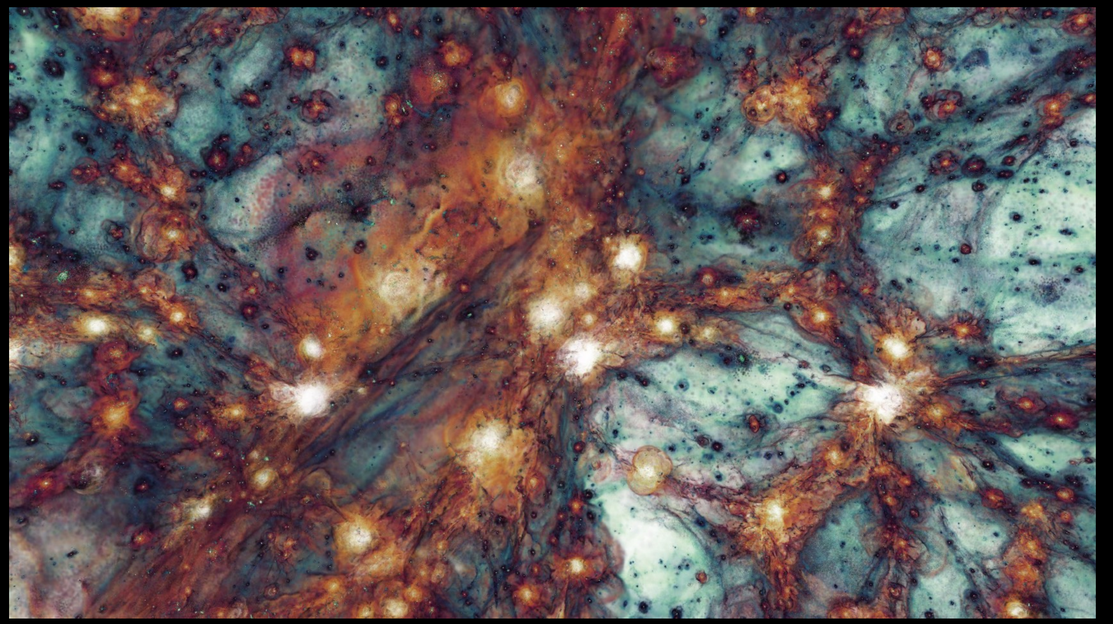




Will Roper

SWIFT

- Hydrodynamics first cosmological simulation code.
- Hybrid task-based parallelism with asynchronous communications.
- Metis graph based domain decomposition.
- Already in use for a diverse range of simulations from Gpc cosmological boxes to planetary impacts.

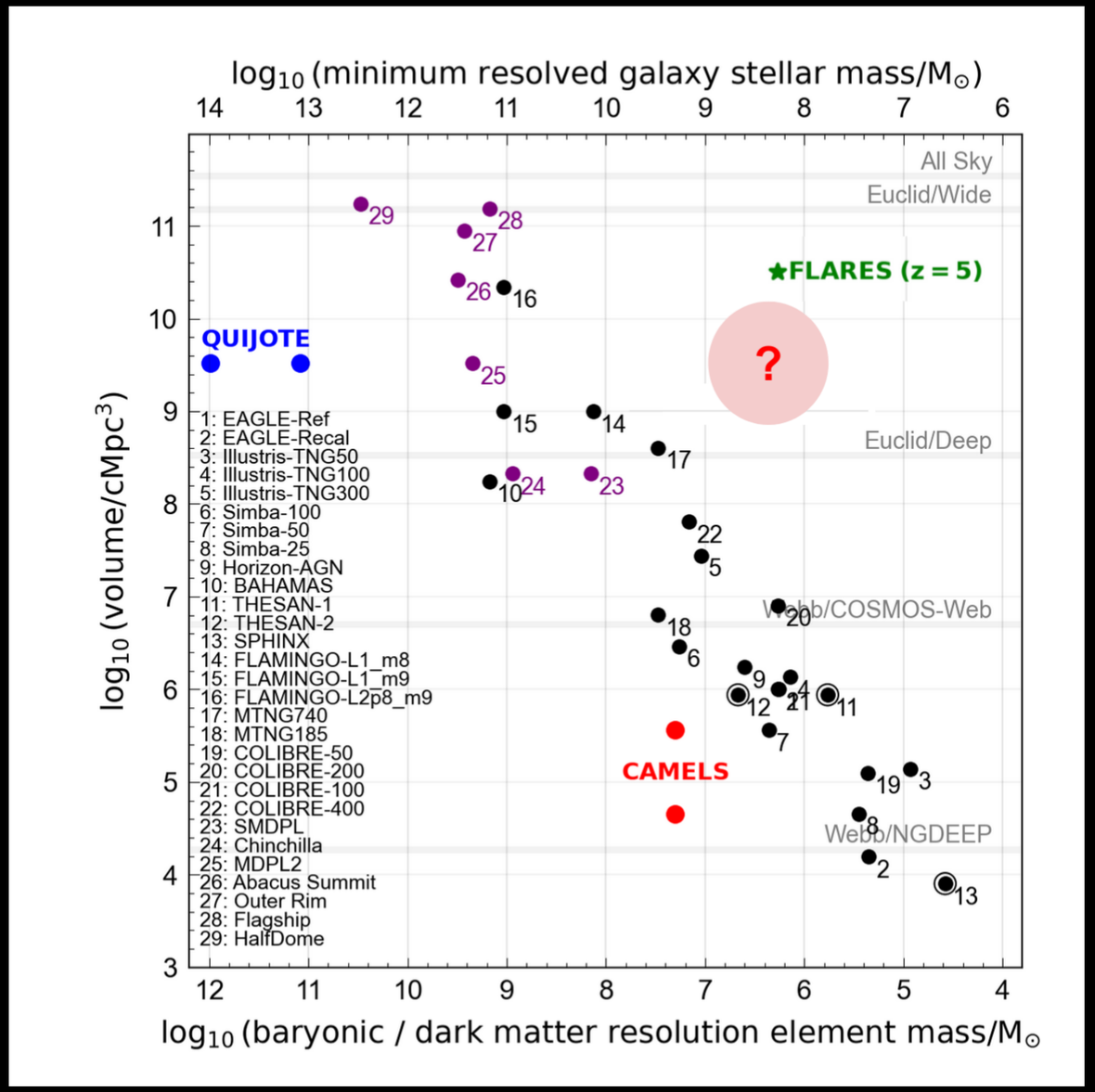


Credit: Stuart McAlpine

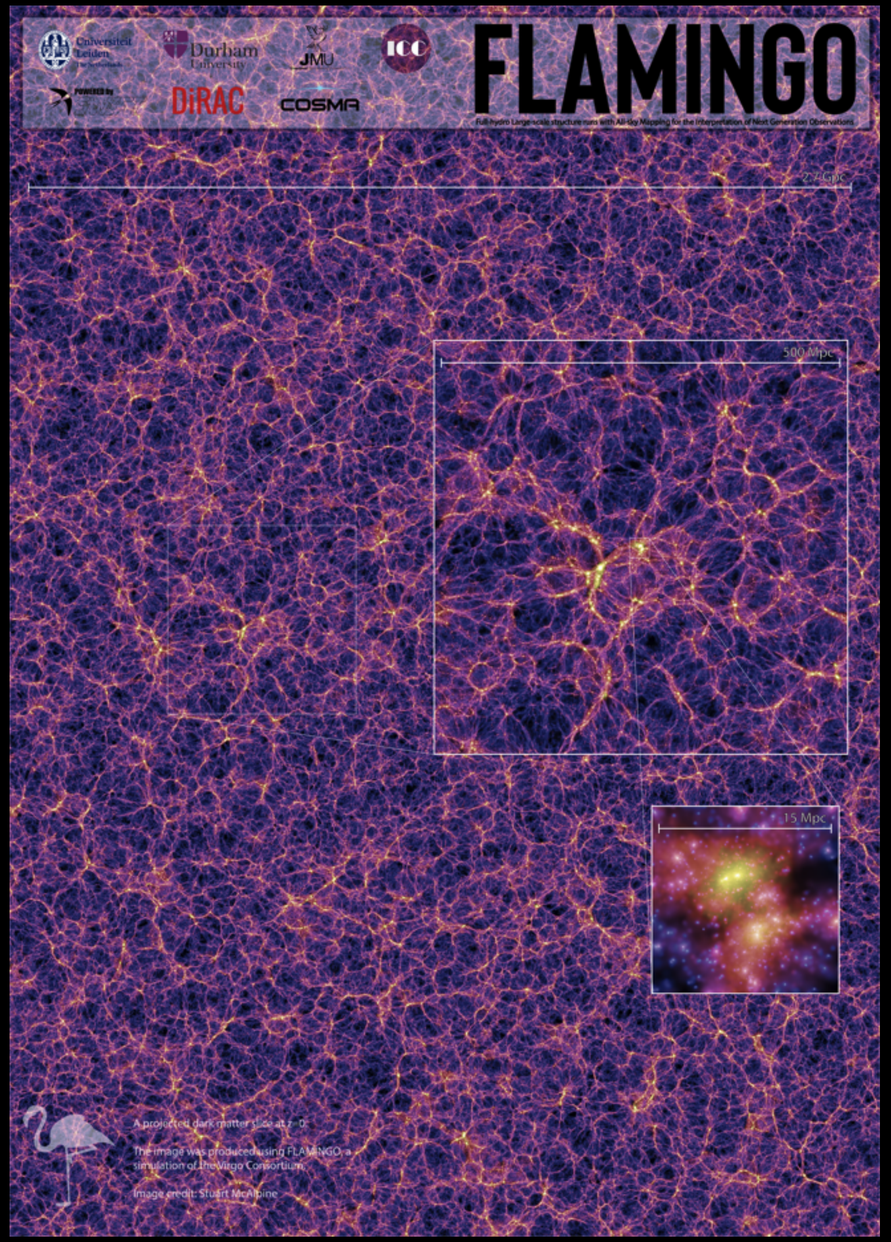


Credit: Jacob Kegerreis

Zooms: Why we NEED them



Credit: Chris Lovell

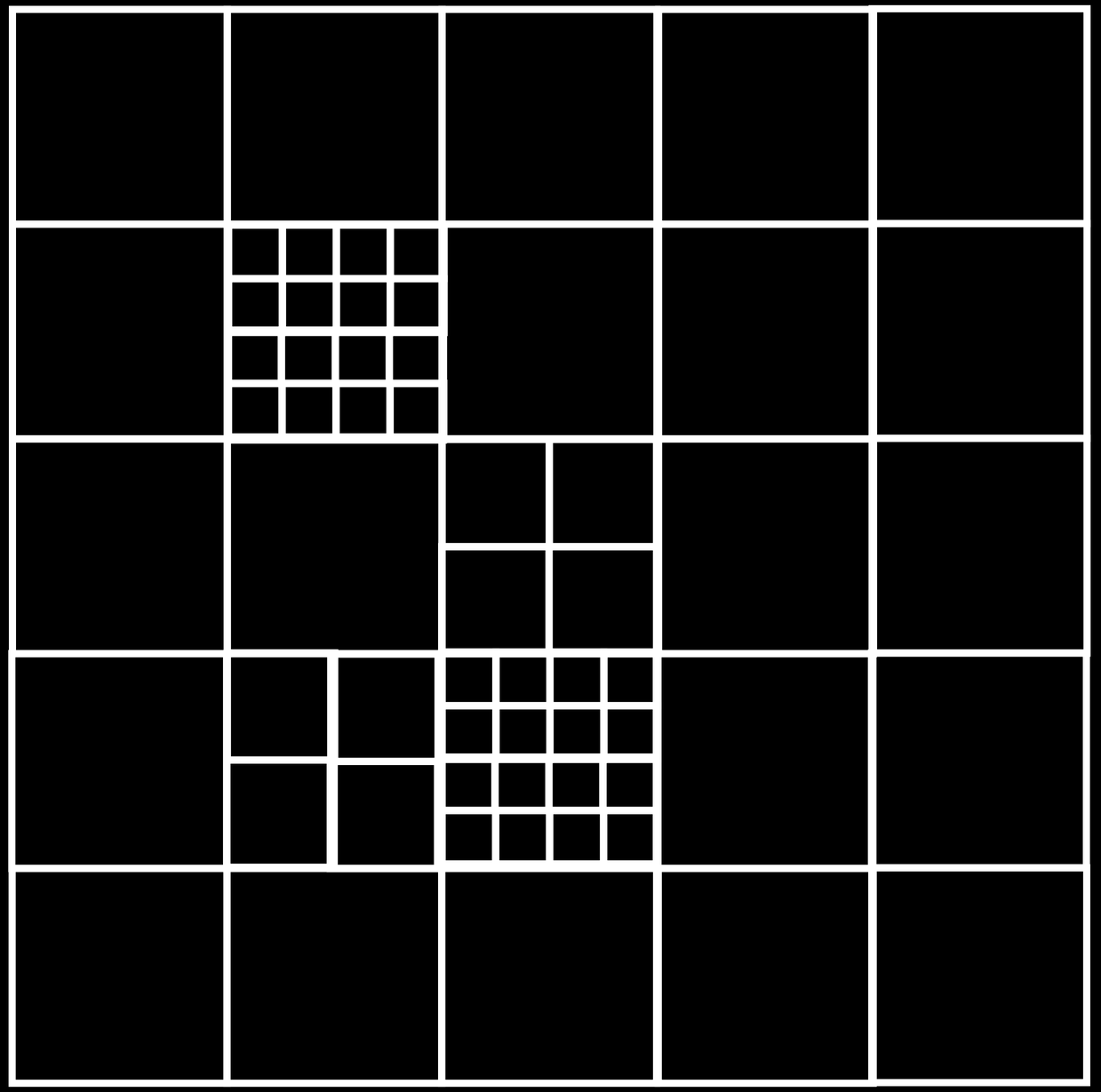


Credit: FLAMINGO project

The Problem

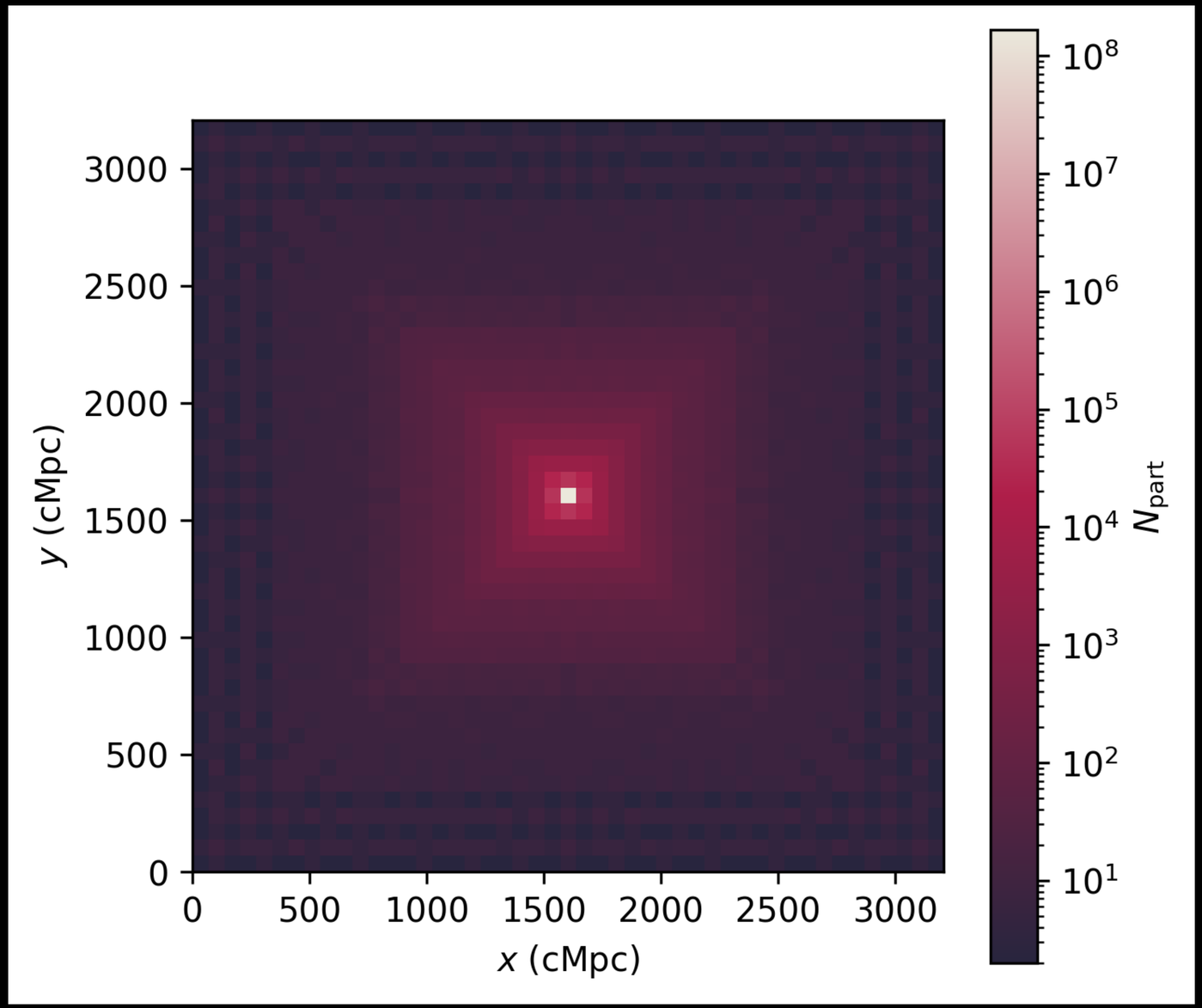
The Balancing Act

- The basic computation elements of SWIFT are cells. Each containing an octree of sub cells.



The Balancing Act

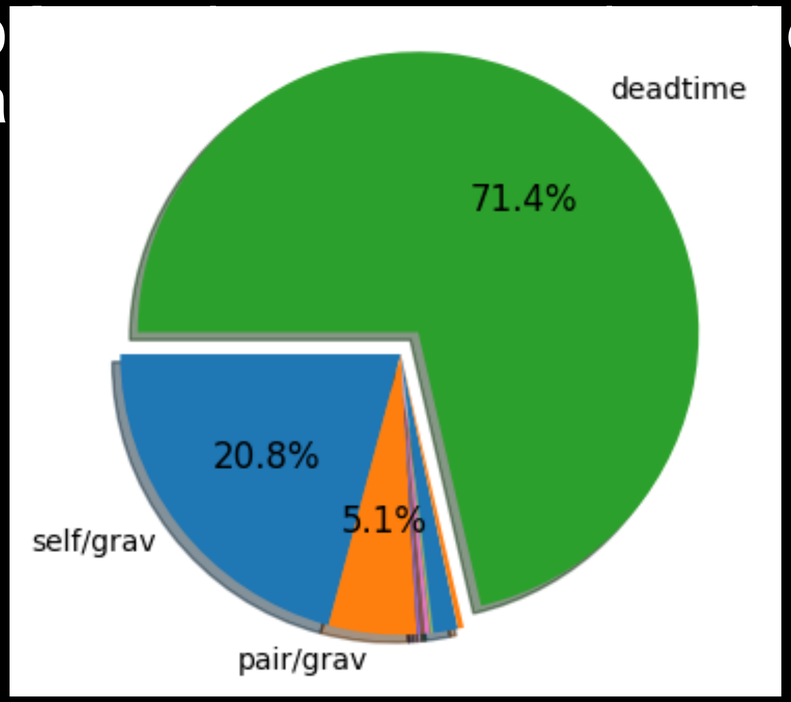
- The basic computation elements of SWIFT are cells. Each containing an octree of sub cells.
- We want uniform division of work. A zoom is the antithesis to this.

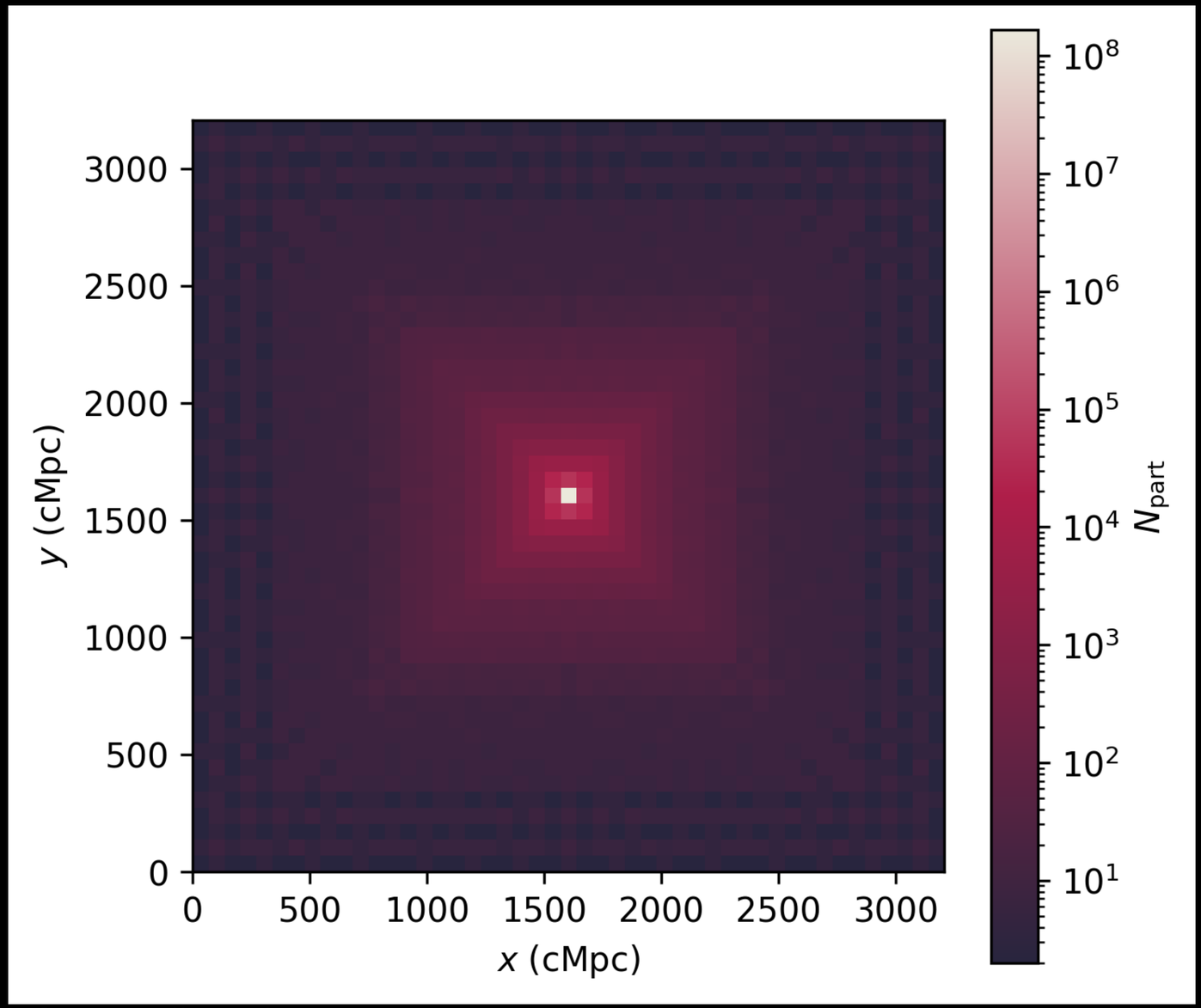


Colormap credit: Josh Borrow, Taylor Swift

The Balancing Act

- The basic computation elements of SWIFT are cells. Each containing an octree of sub cells.
- We want uniform division o[illegible]e a[illegible]

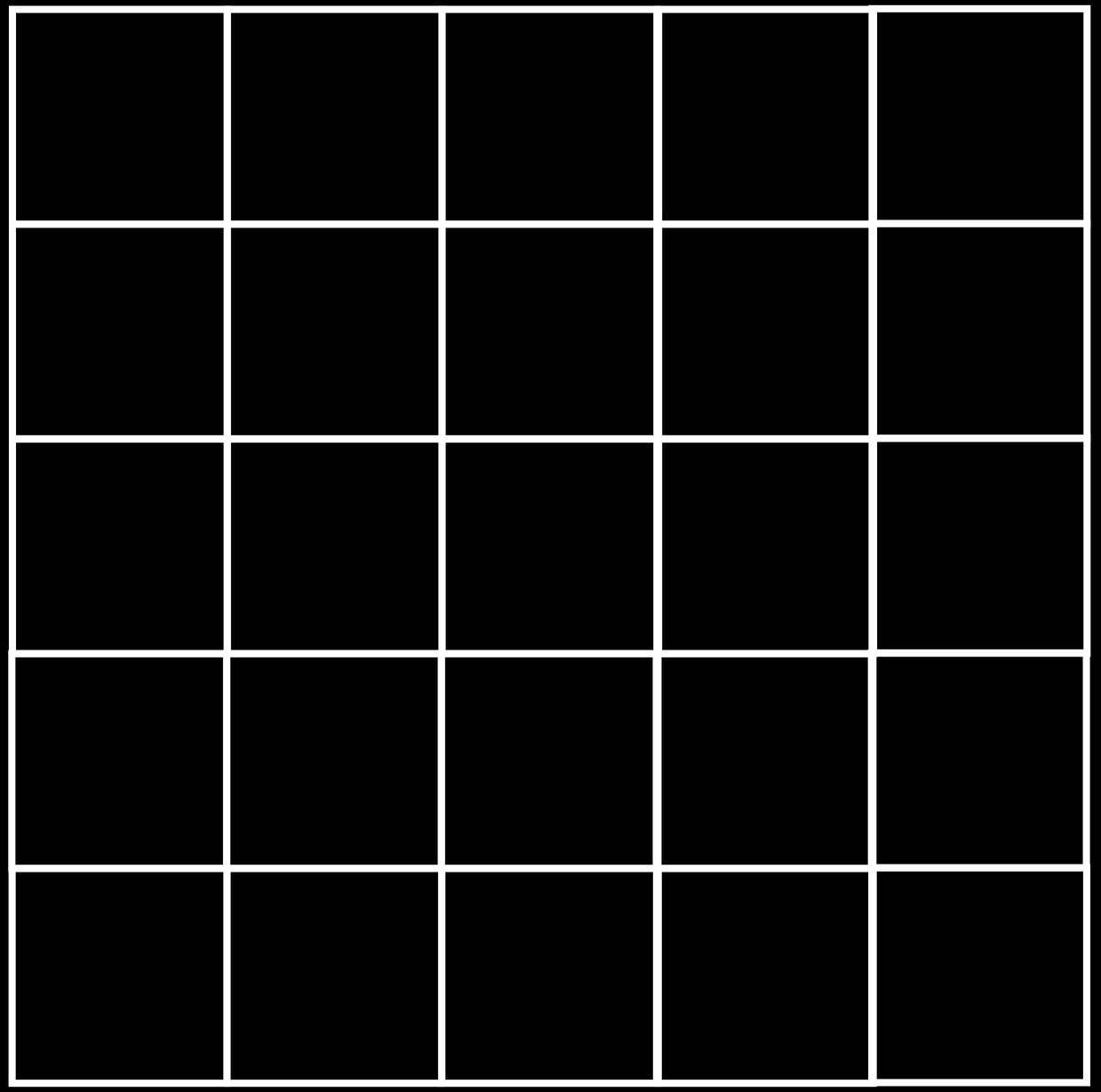




Colormap credit: Josh Borrow, Taylor Swift

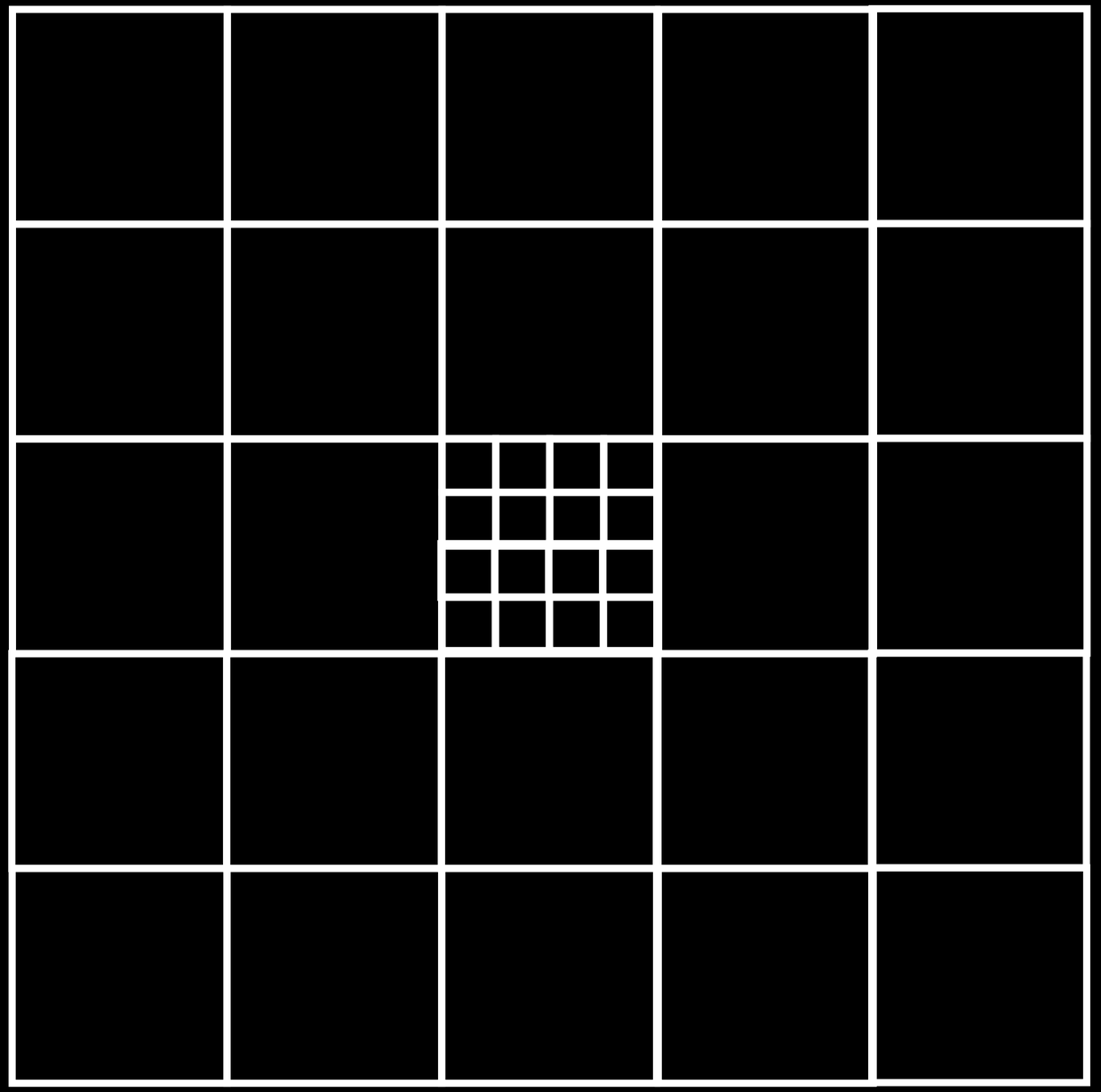
The Balancing Act

- The basic computation elements of SWIFT are cells.
- We want uniform division of work. A zoom is the antithesis to this.
- Goal: divided up the work better.

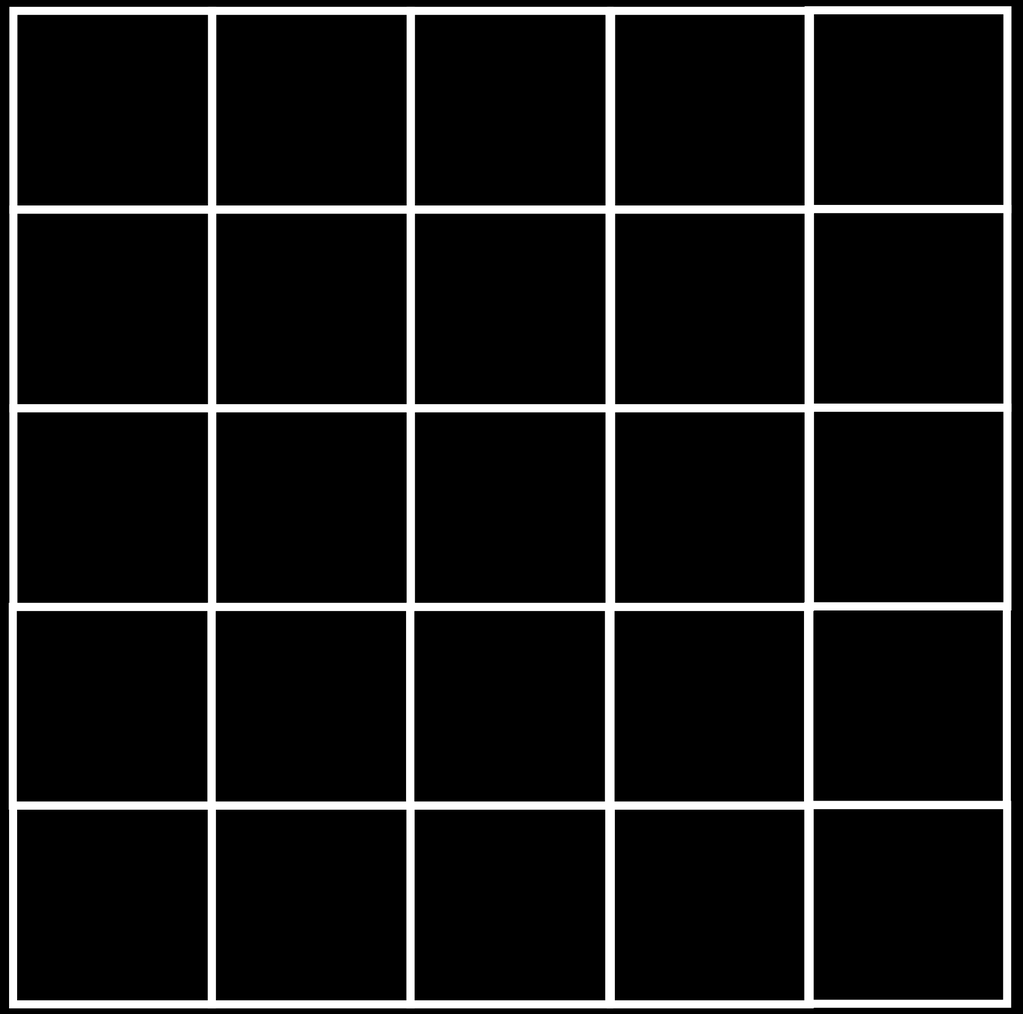


The Balancing Act

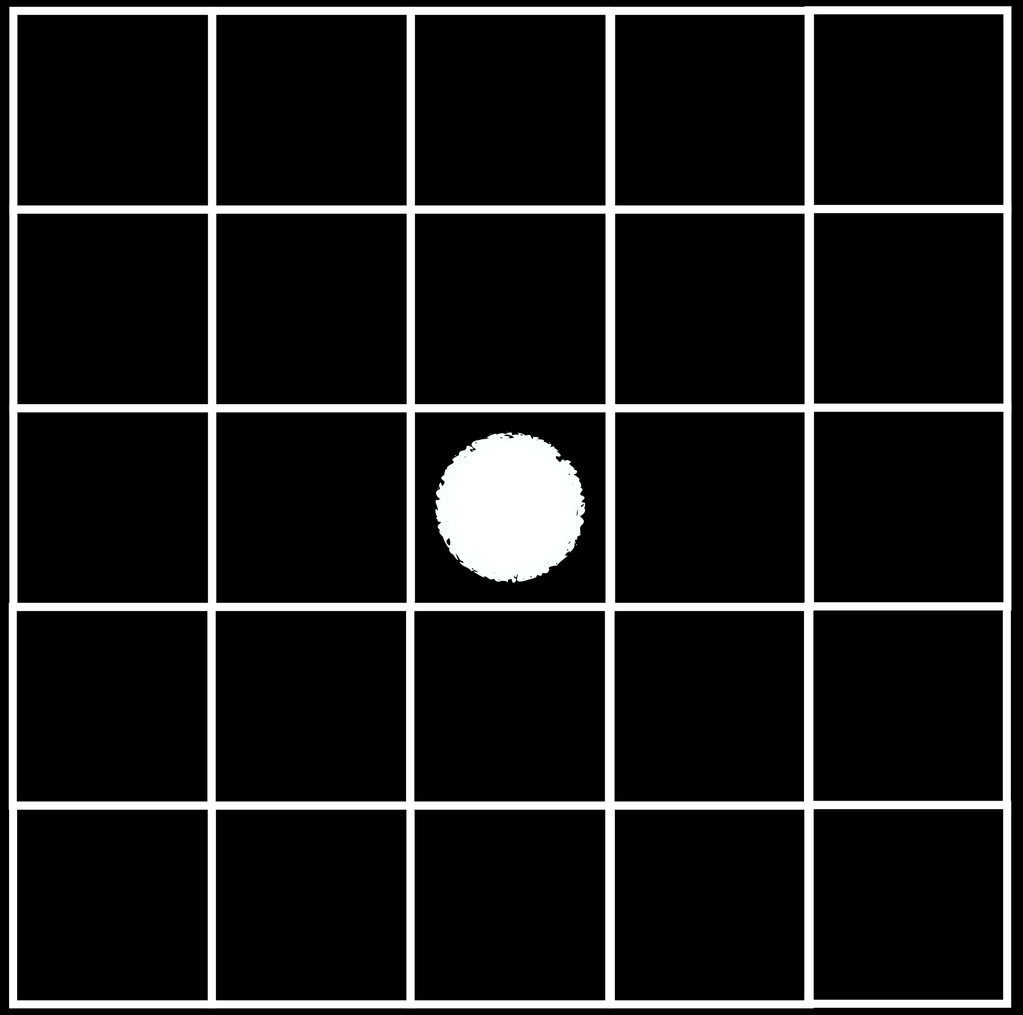
- The basic computation elements of SWIFT are cells.
- We want uniform division of work. A zoom is the antithesis to this.
- Goal: divided up the work better.
- Solution: second grid of top level zoom cells.



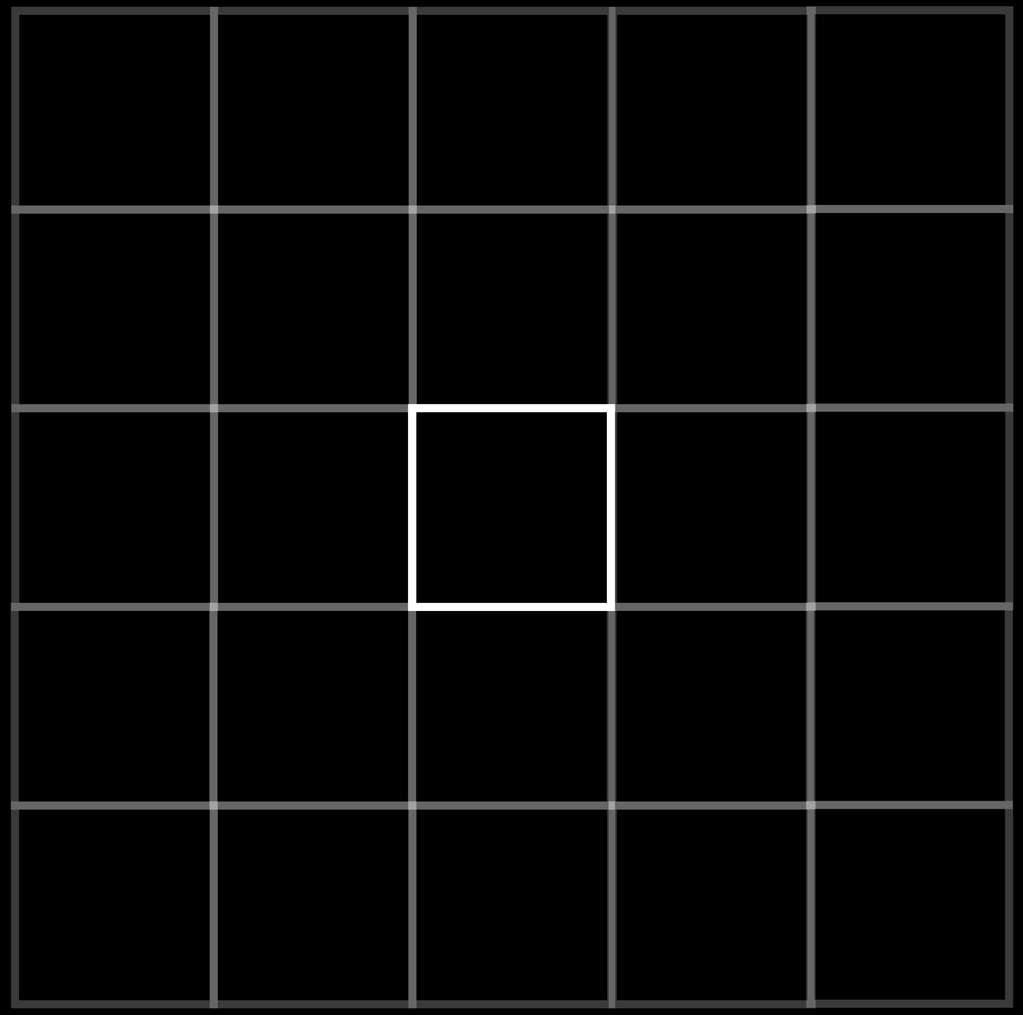
Cell Construction

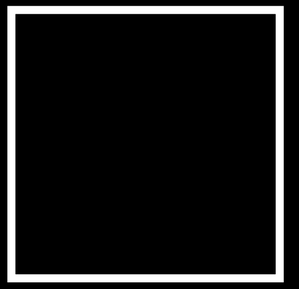


Cell Construction



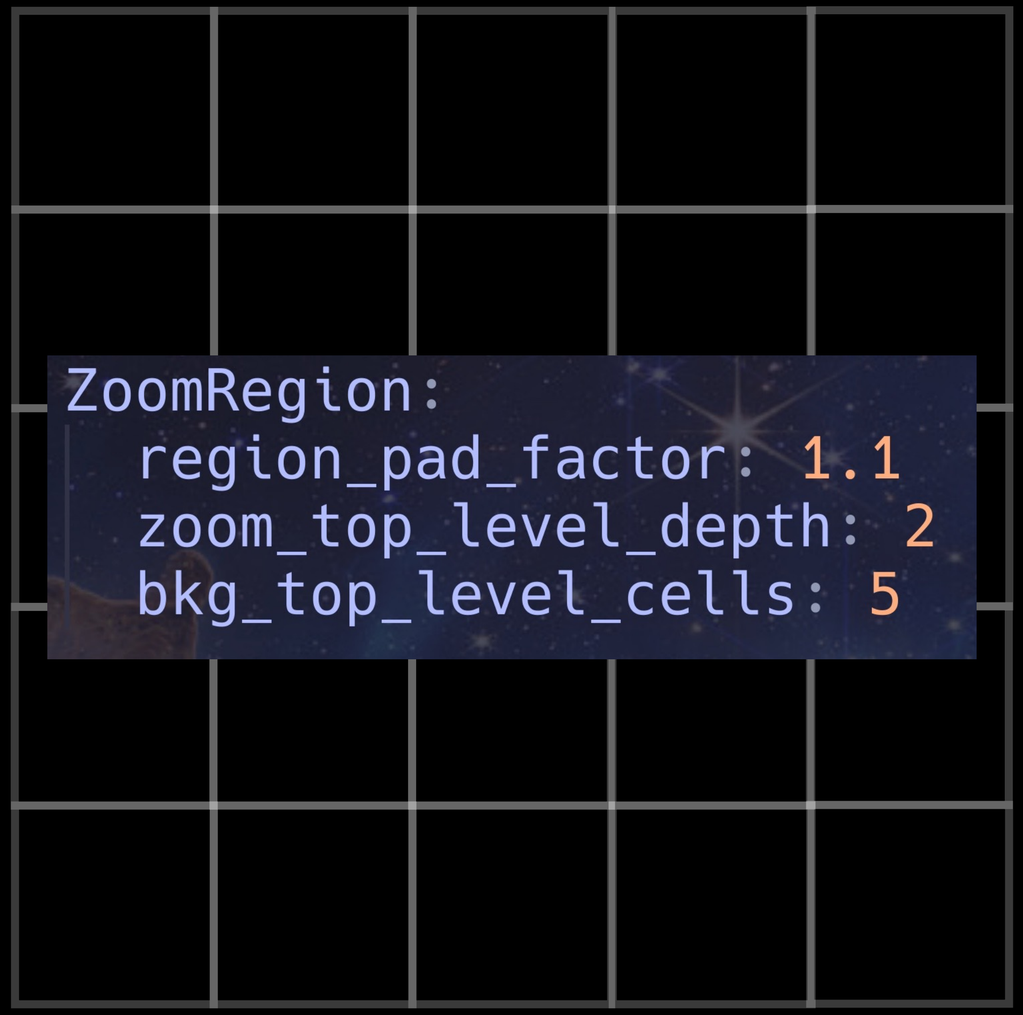
Cell Construction



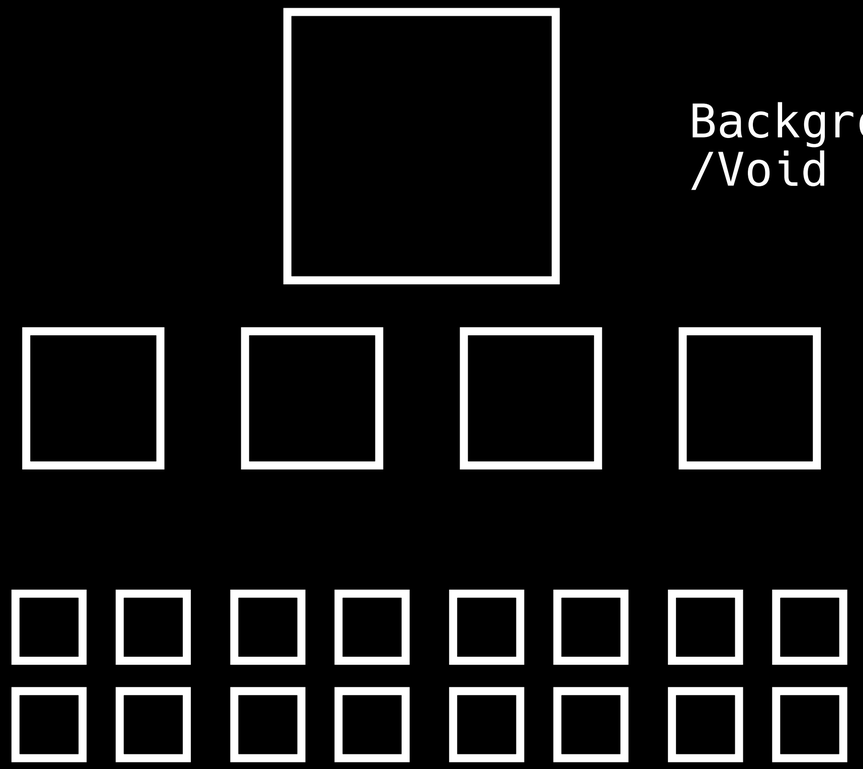


Background
/Void

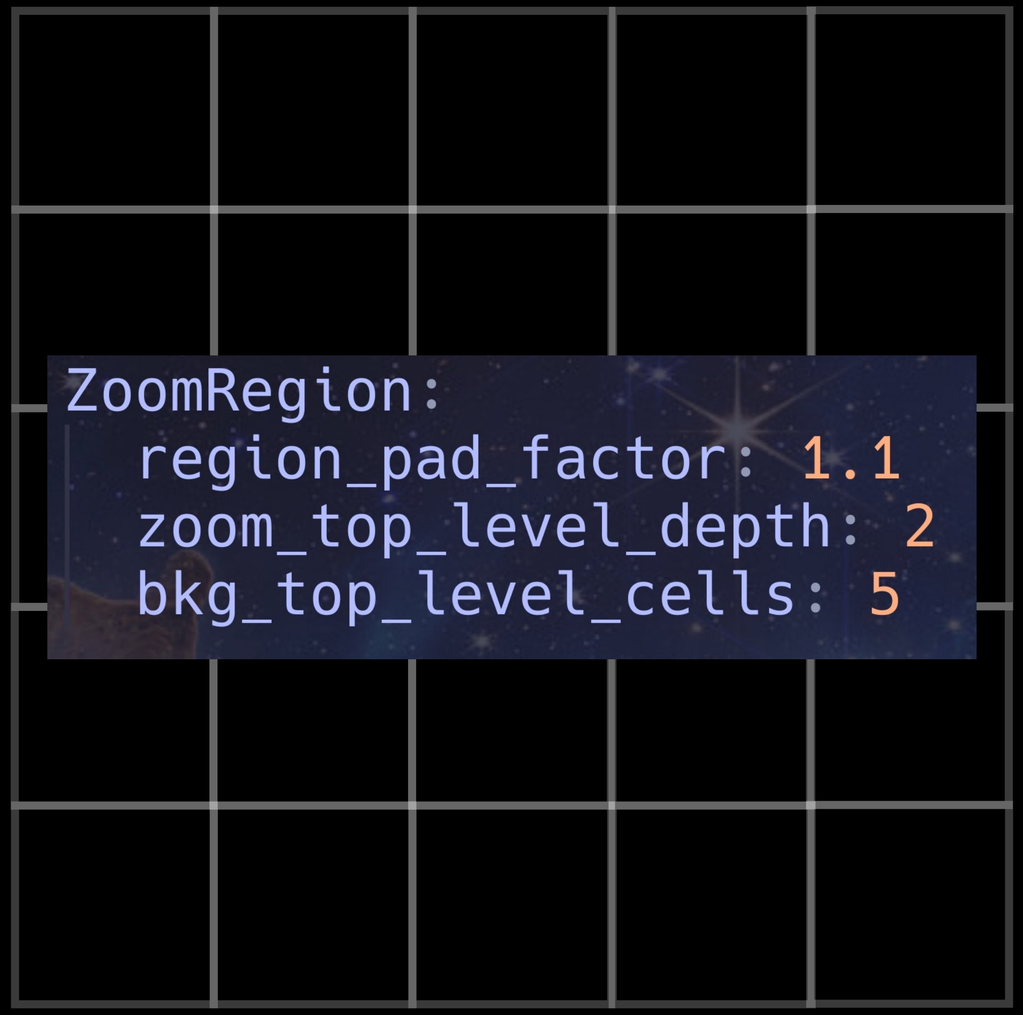
Cell Construction

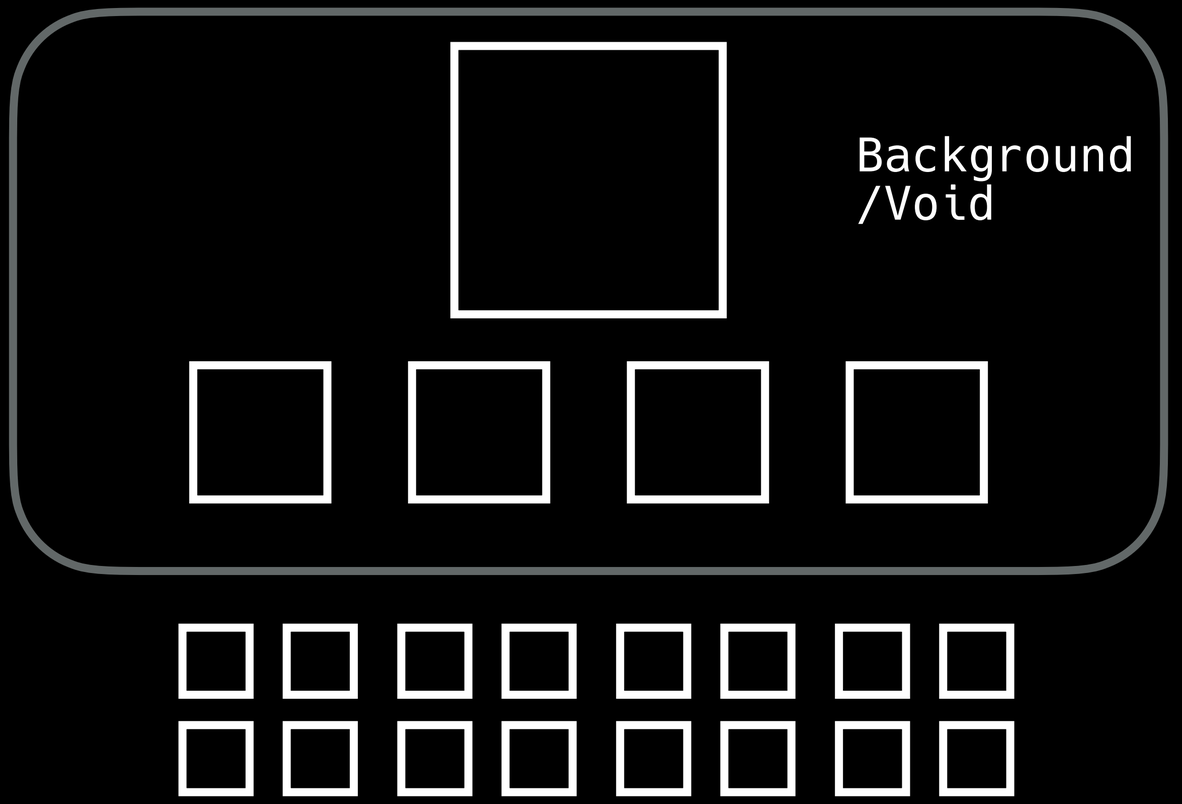


```
ZoomRegion:
  region_pad_factor: 1.1
  zoom_top_level_depth: 2
  bkg_top_level_cells: 5
```

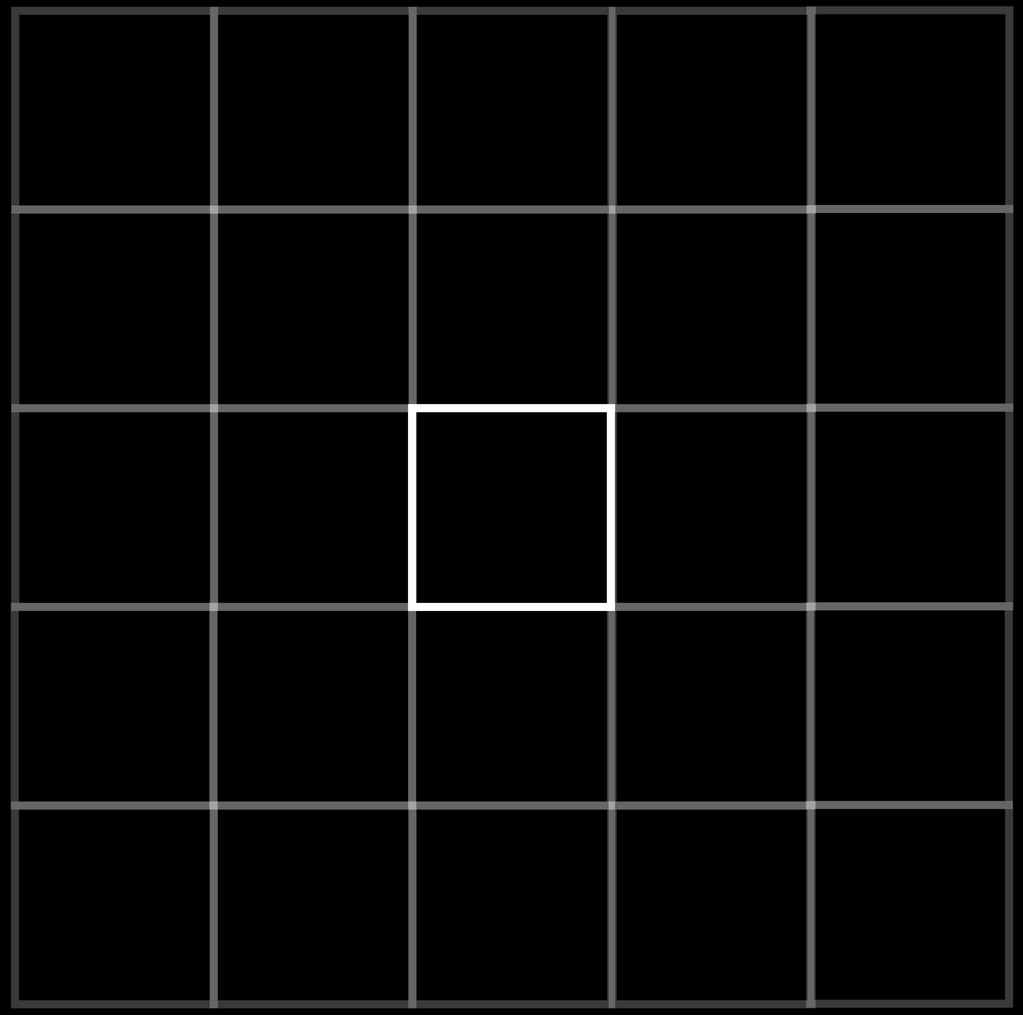


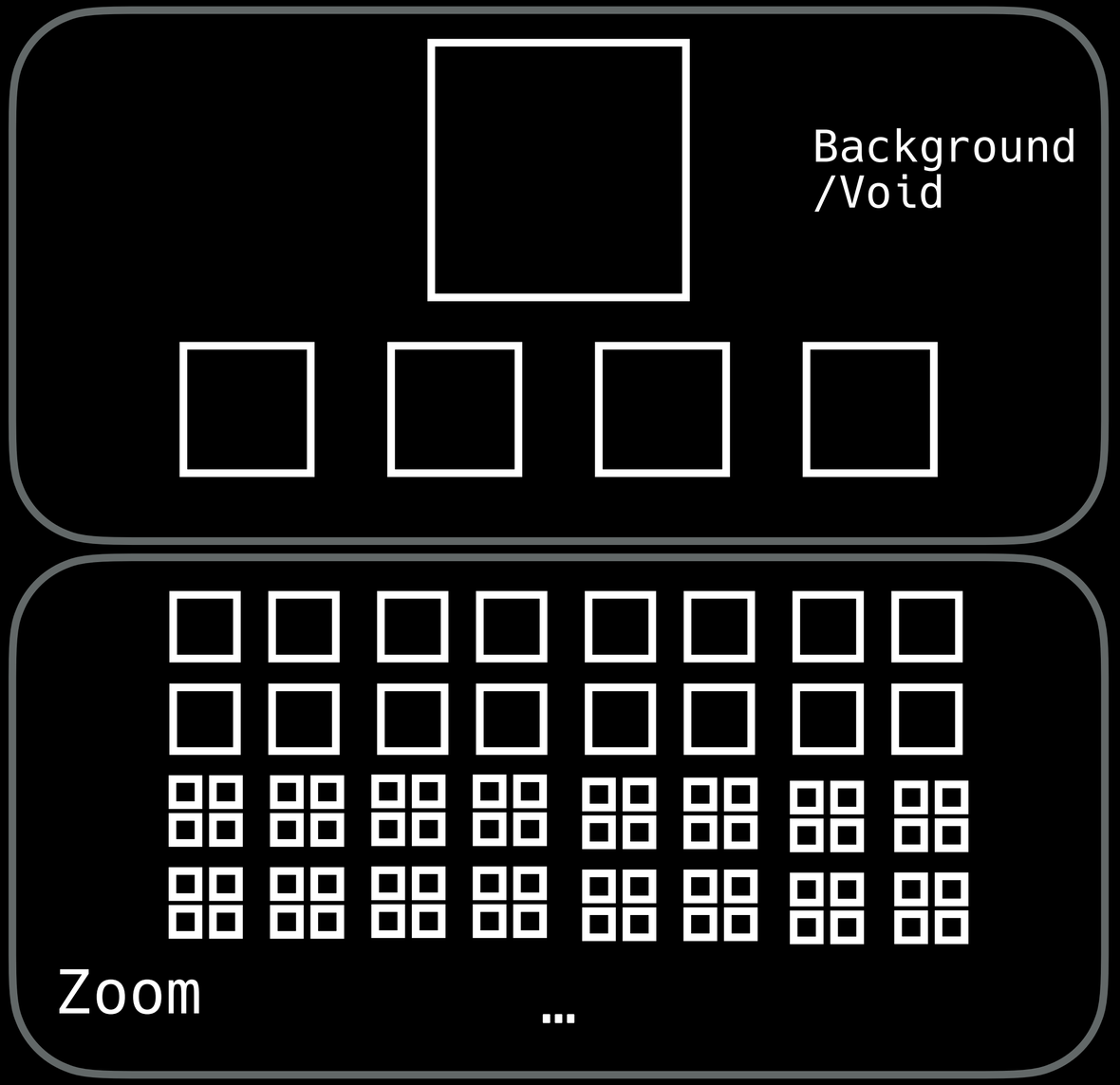
Cell Construction



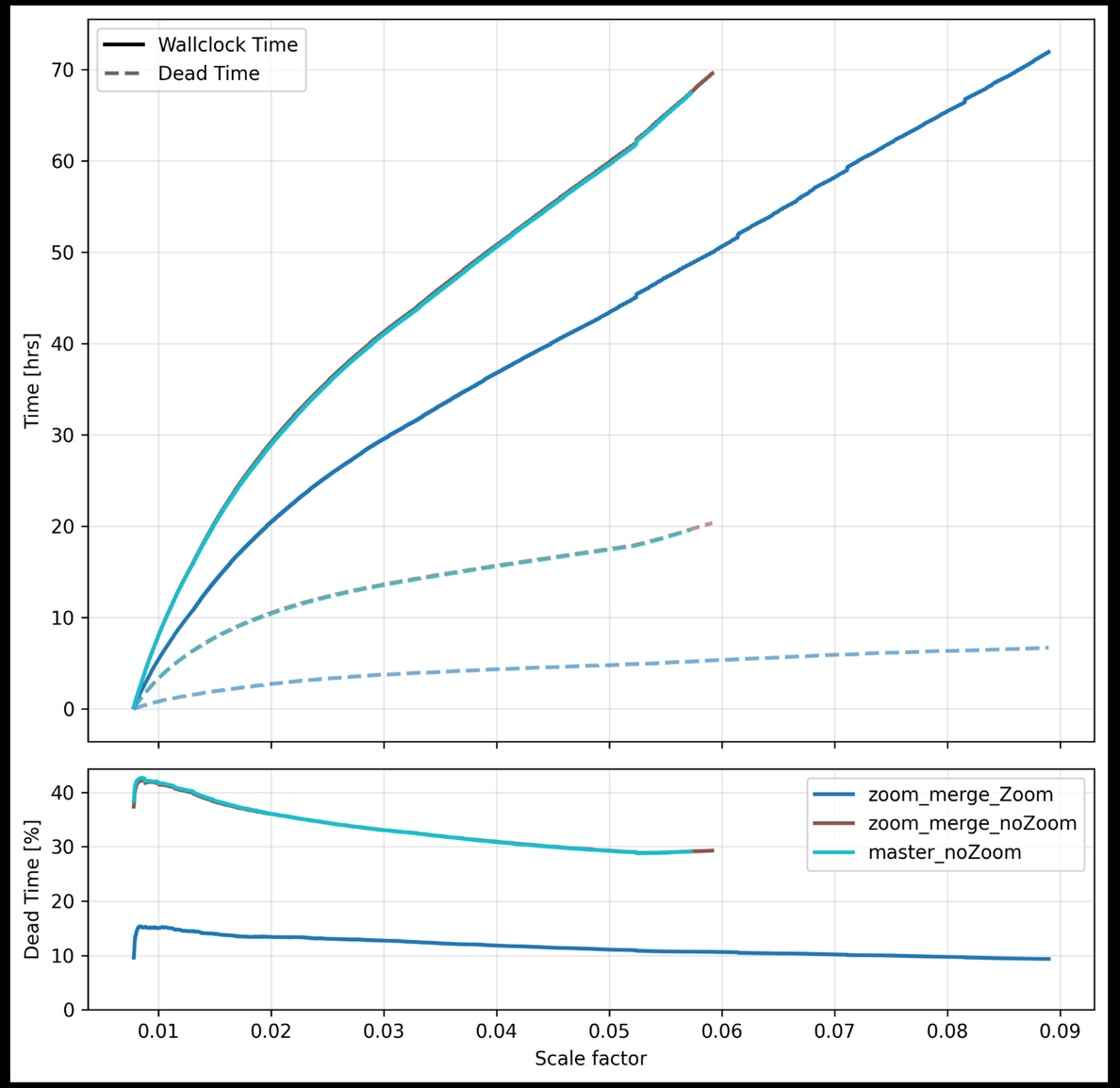


Cell Construction



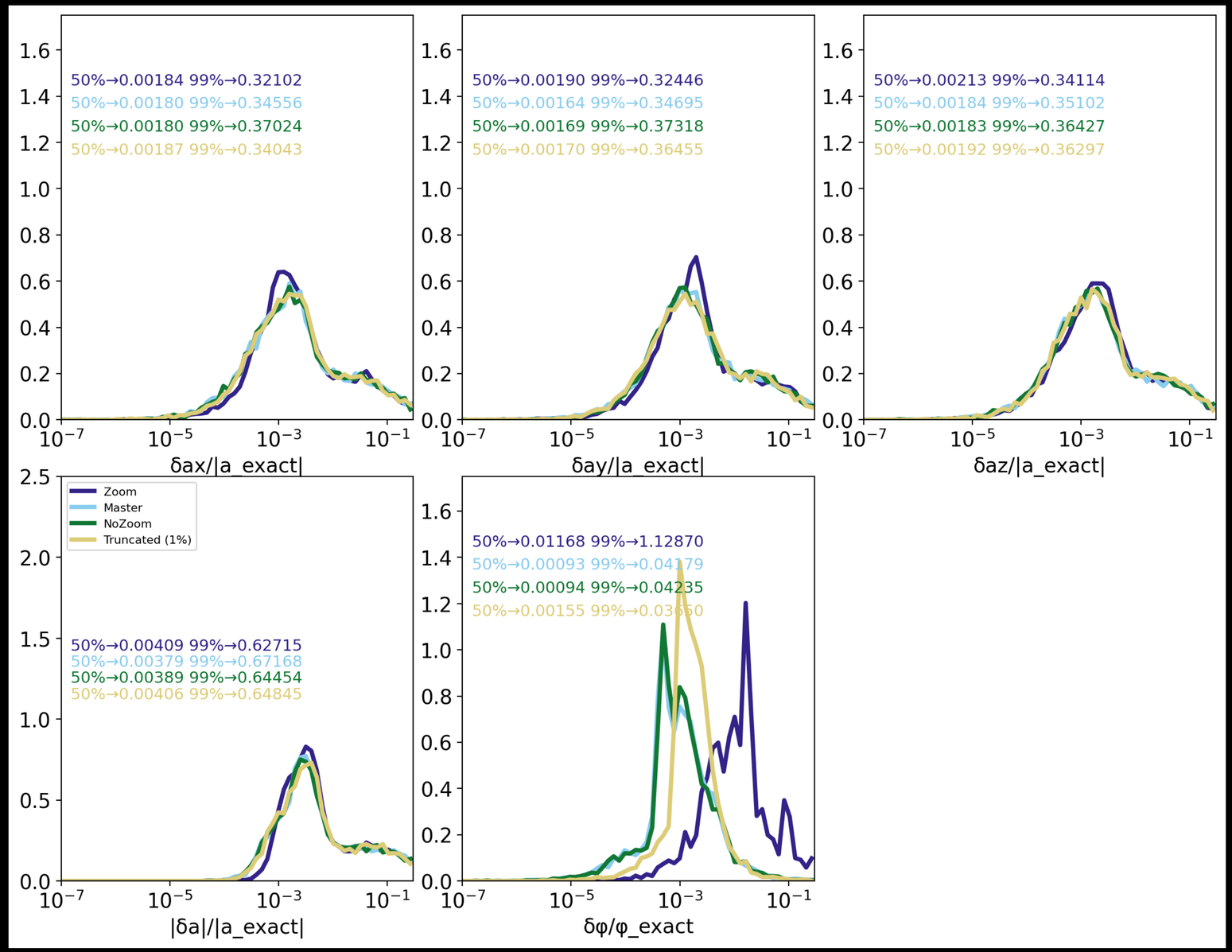


First Test: Does the job

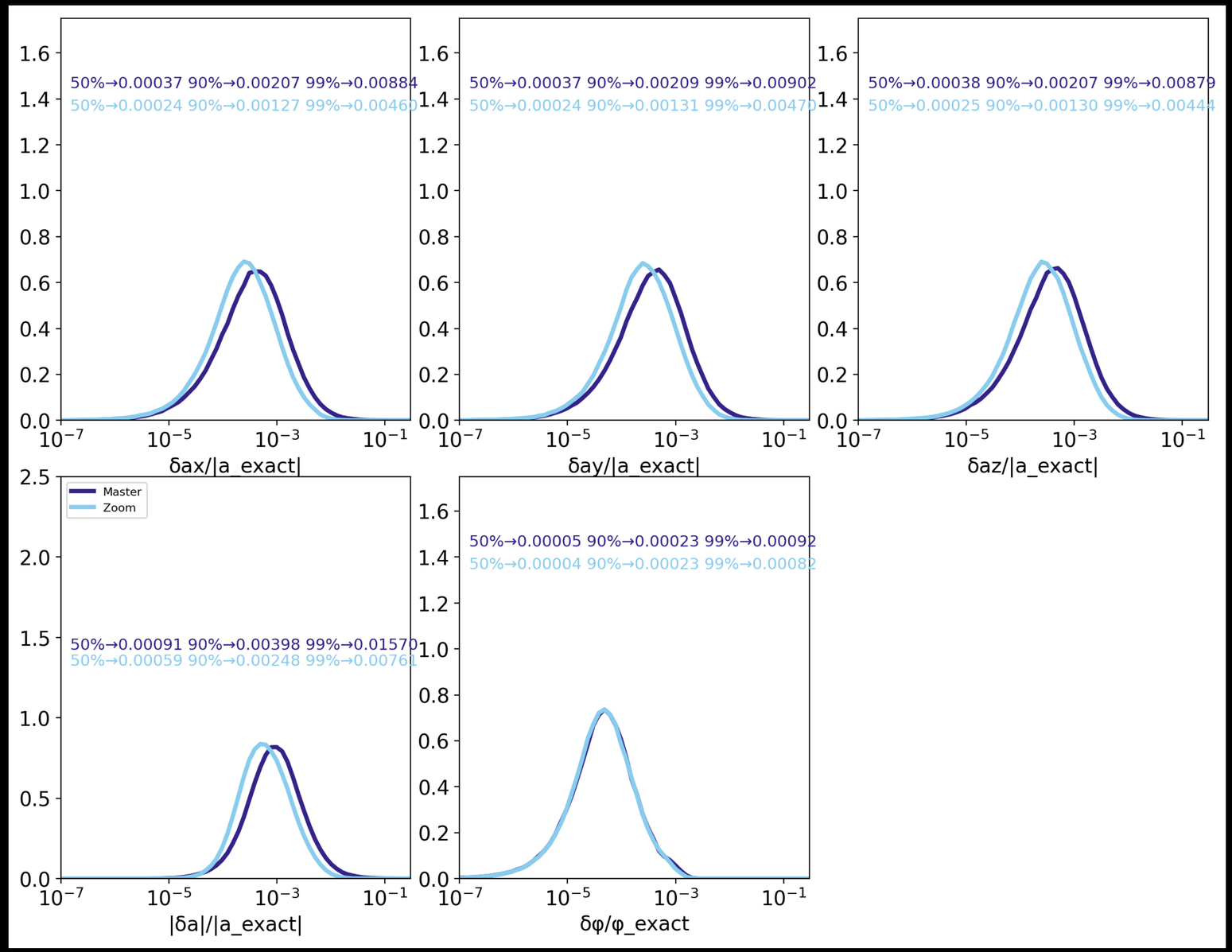


Biting The Bullet: Are we correct still?

Checking Accuracy (A Success?)



Checking Accuracy (A Success!)



Optimising Everything At My Own Expense

Gravity Regimes

Particle-Particle

- Short range
- Direct summation over particles.
- Expensive.
- Technically, also Particle-Multipole where possible (cheaper).

Multipole-Multipole

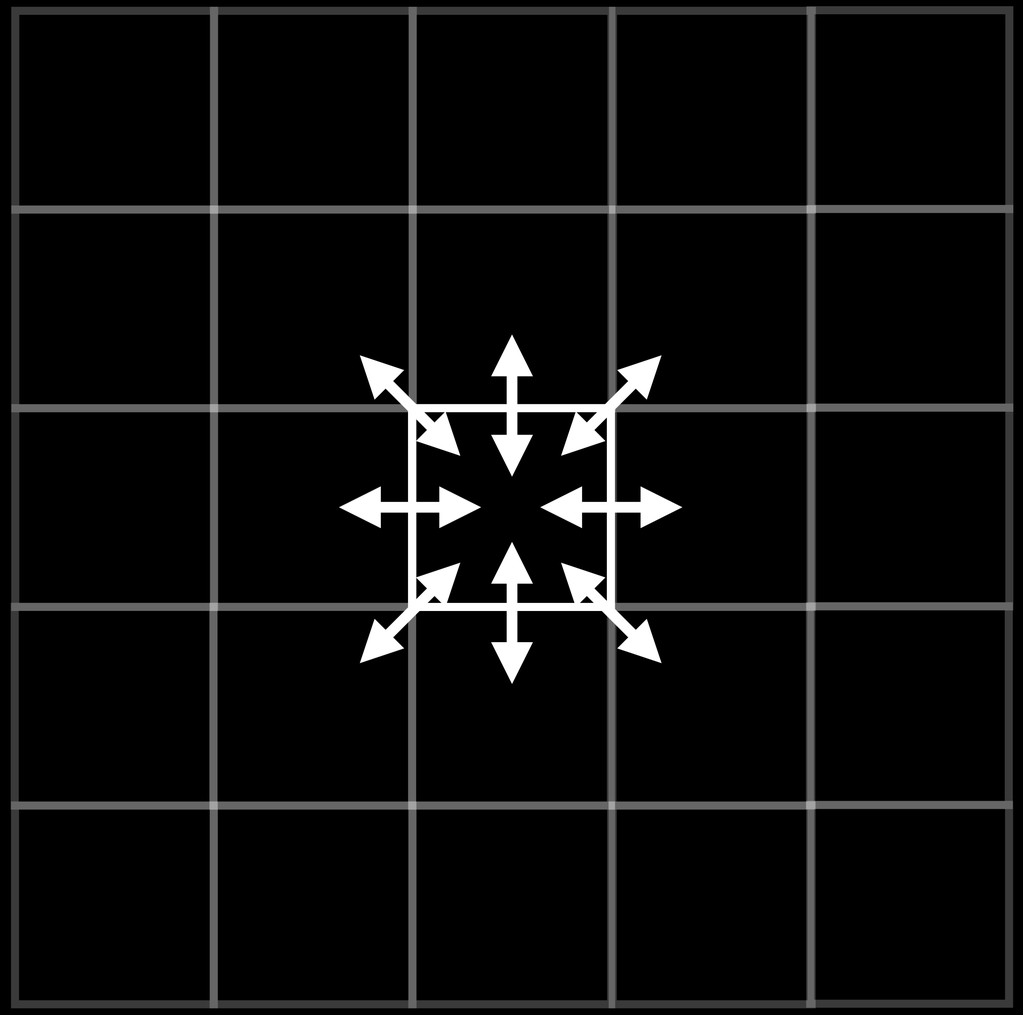
- Mid-long range.
- Interaction between multipoles in cells (FMM).
- Cheaper.

Particle-Mesh

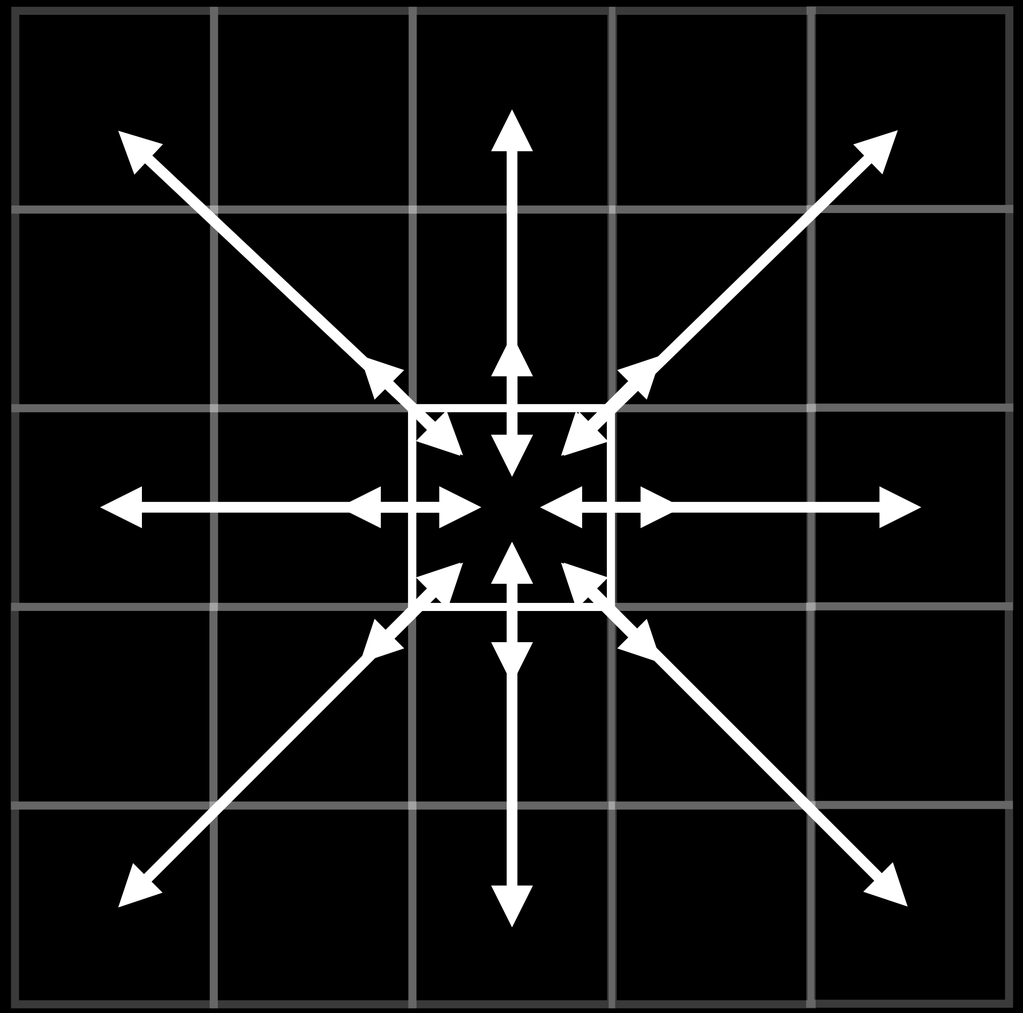
- Long range (periodic) interactions.
- Cheapest.
- Higher resolution mesh -> less PP and MM.



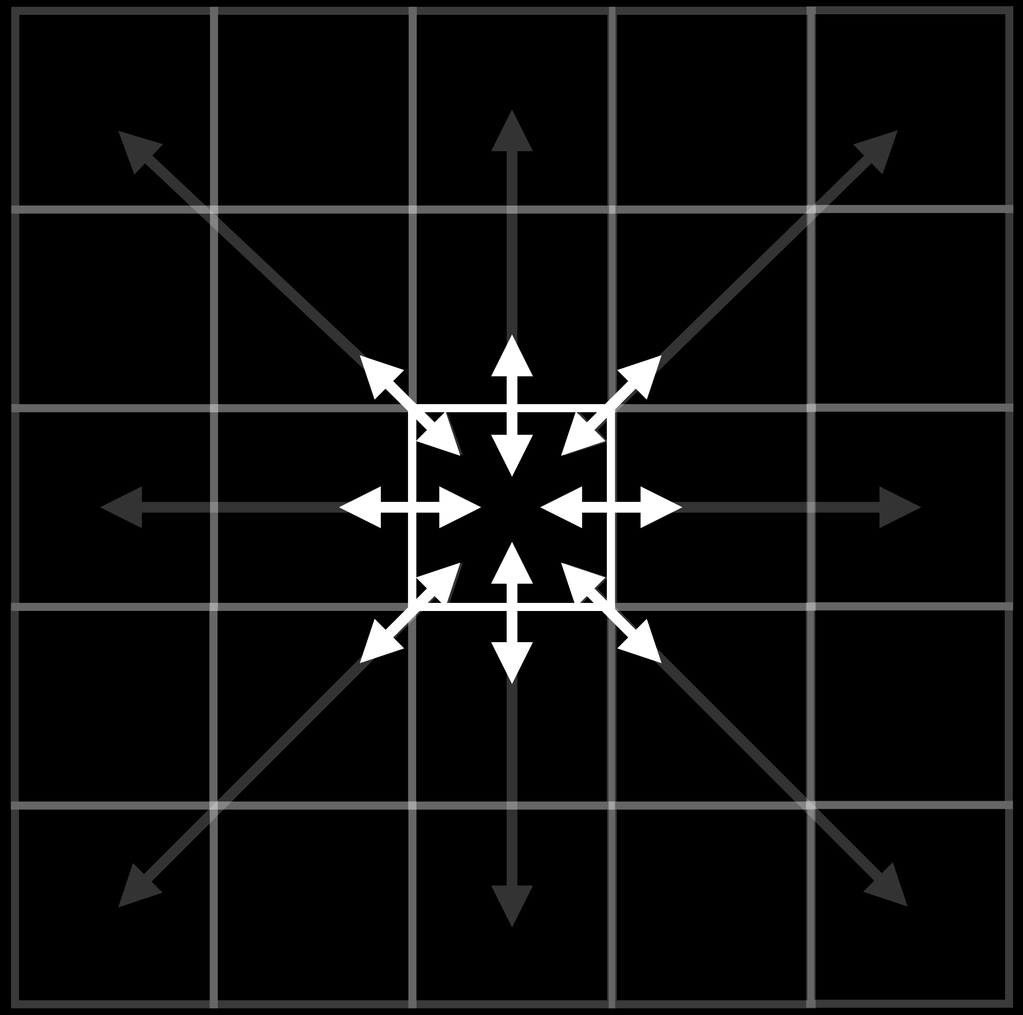
Gravity pair tasks

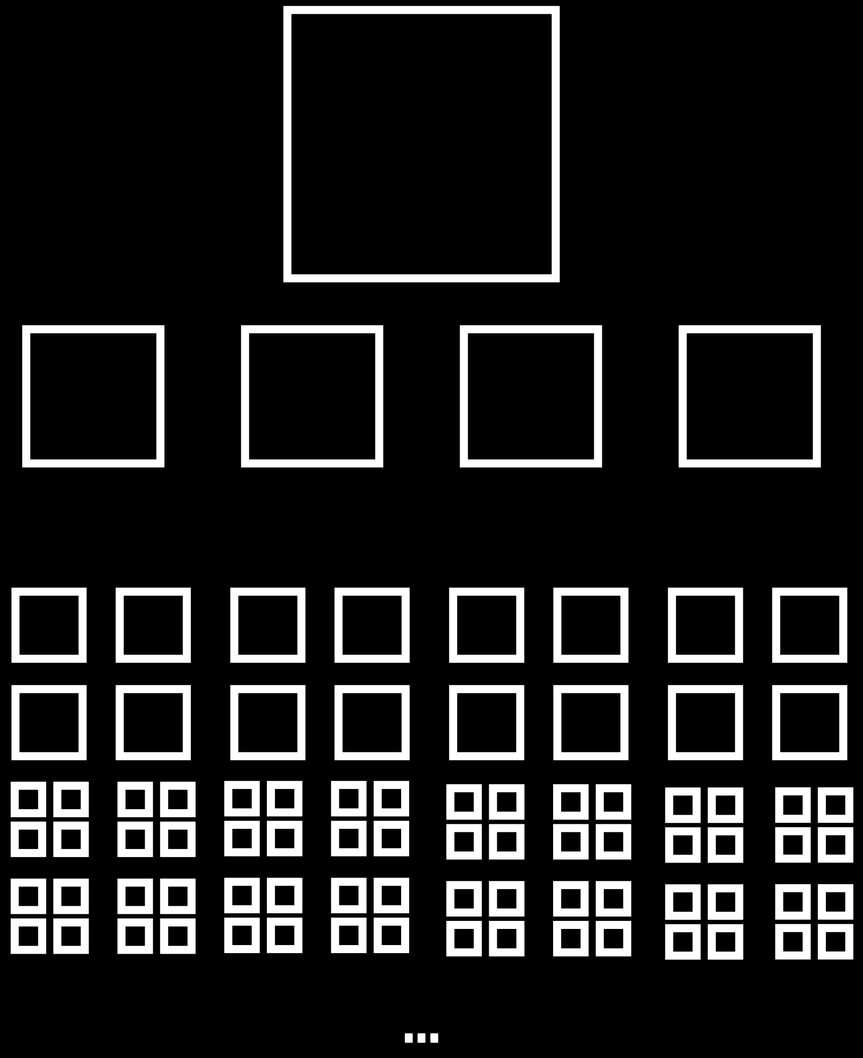


Gravity pair tasks

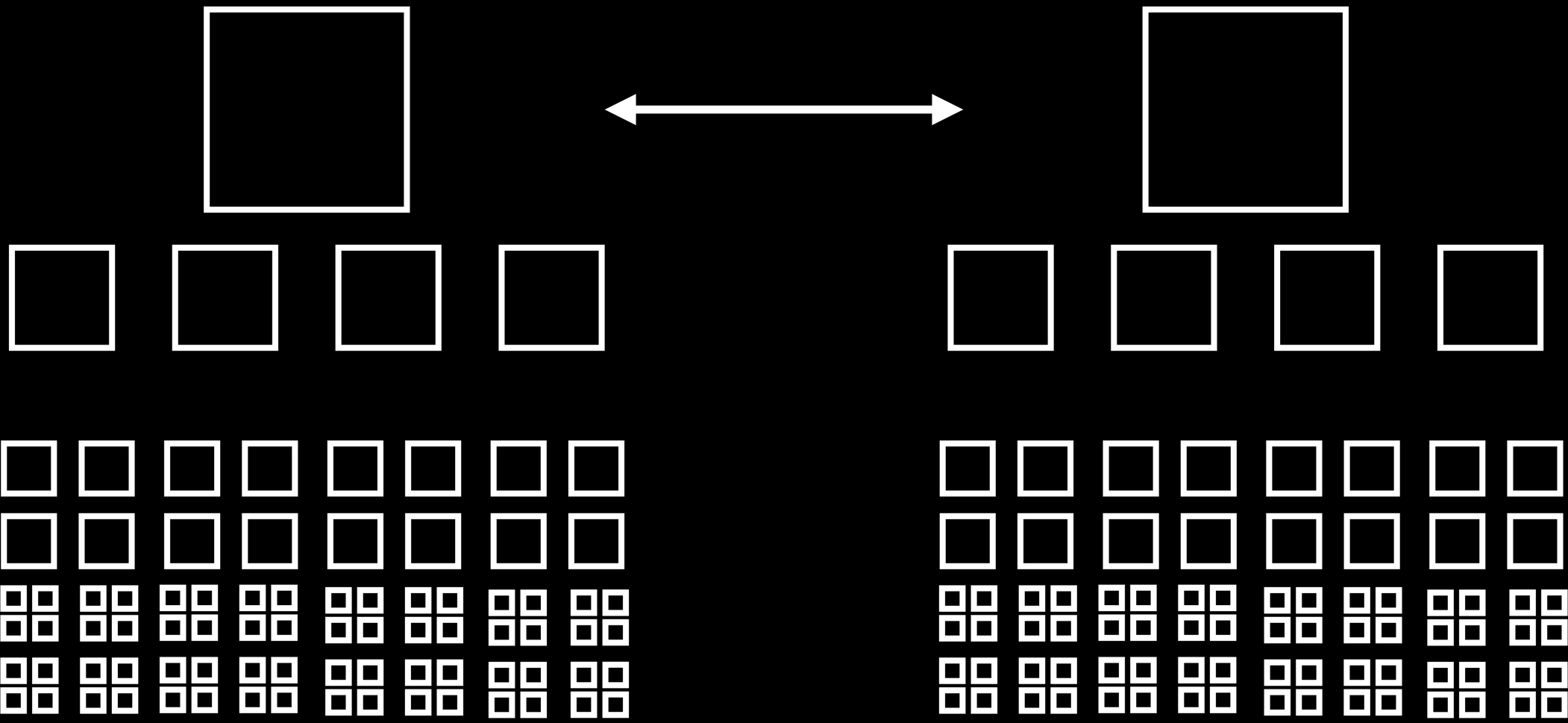


Gravity pair tasks

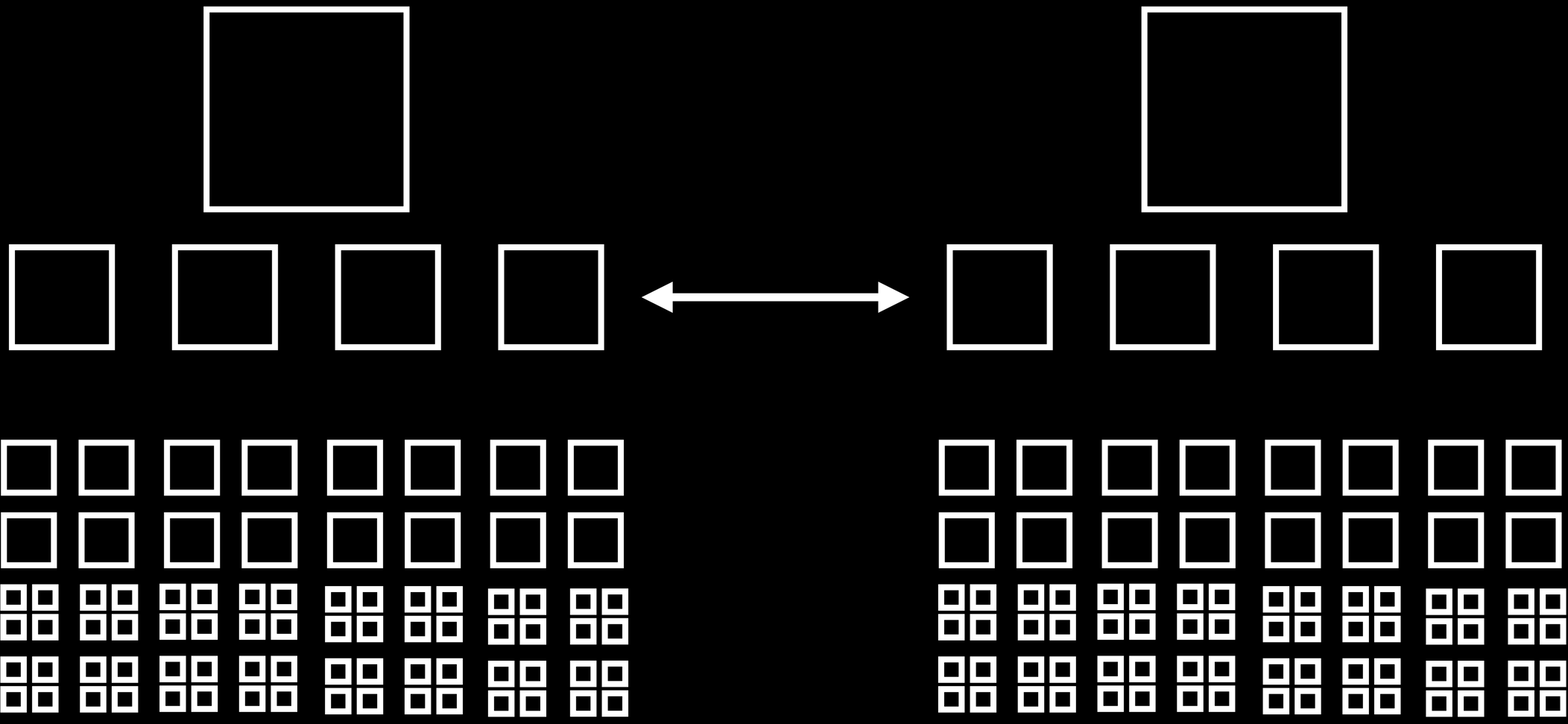




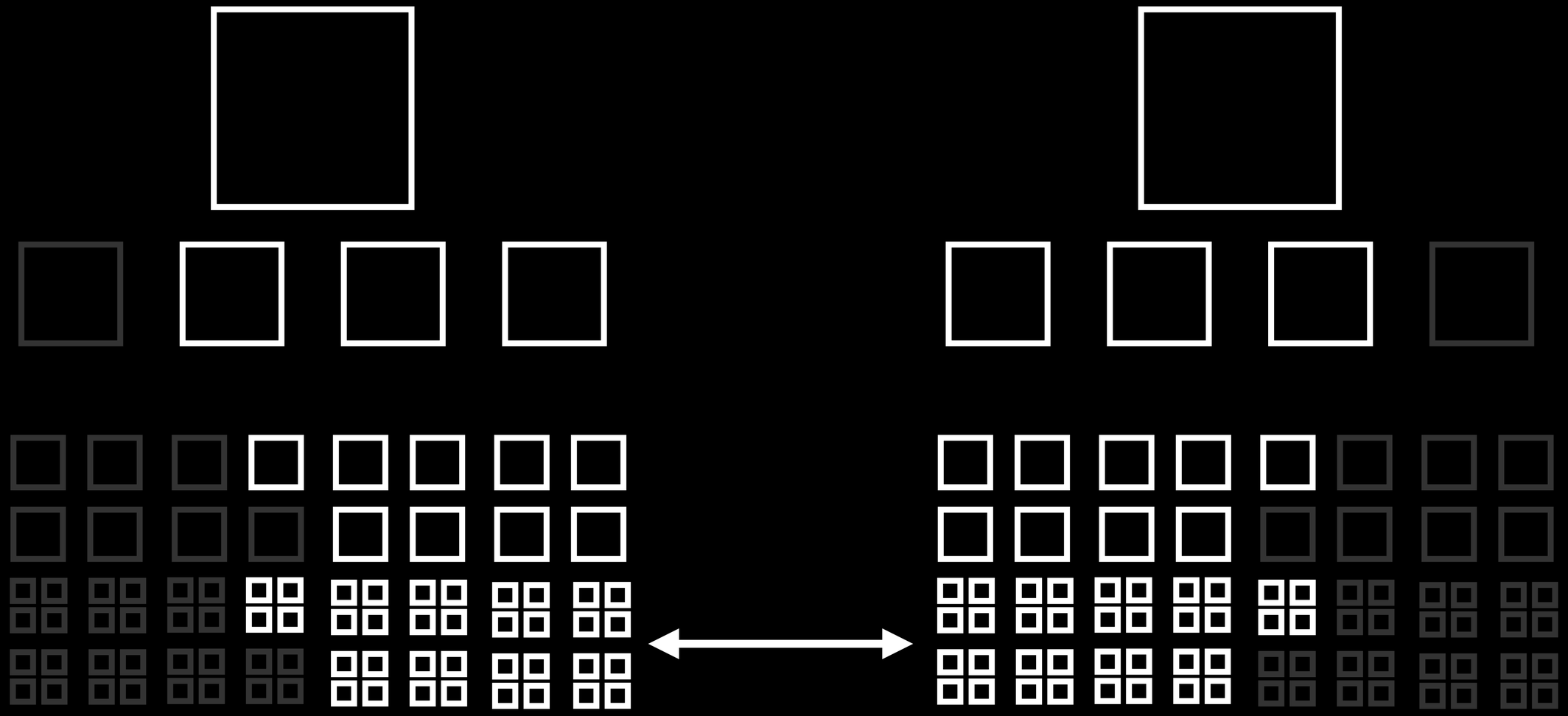
Gravity pair task splitting



Gravity pair task splitting



Gravity pair task splitting



We All Love Mesh Gravity

No mesh considerations during splitting:

```
[00195.0] engine_print_task_counts: pair task type counts are [ zoom<->zoom=229117  zoom<->bkg=612831 bkg<->bkg=53068 zoom<->neighbour=612831 ]
```

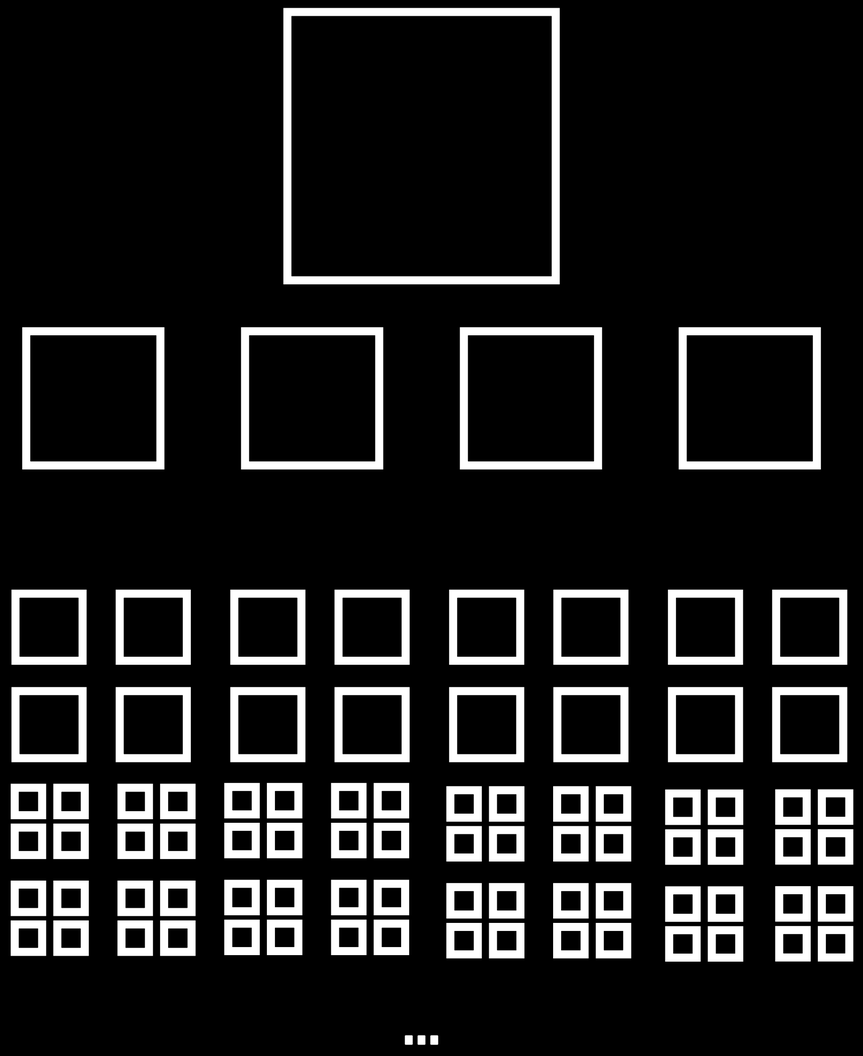
Considering mesh distance for zoom top level pairs:

```
[00195.3] engine_print_task_counts: pair task type counts are [ zoom<->zoom=33824  zoom<->bkg=612831 bkg<->bkg=53068 zoom<->neighbour=612831 ]
```

- Consider the mesh while splitting tasks.
- Removes tasks that immediately exit.
- Results in ~90% reduction in pairs.
- Affects every SWIFT run, with up to 25% runtime improvement.

Cell Construction Dominating

- Highly optimised DFS cell construction is great… most of the time.
- Zooms predisposed to imbalance trees leads to poor parallelisation.



Cell Construction Dominating

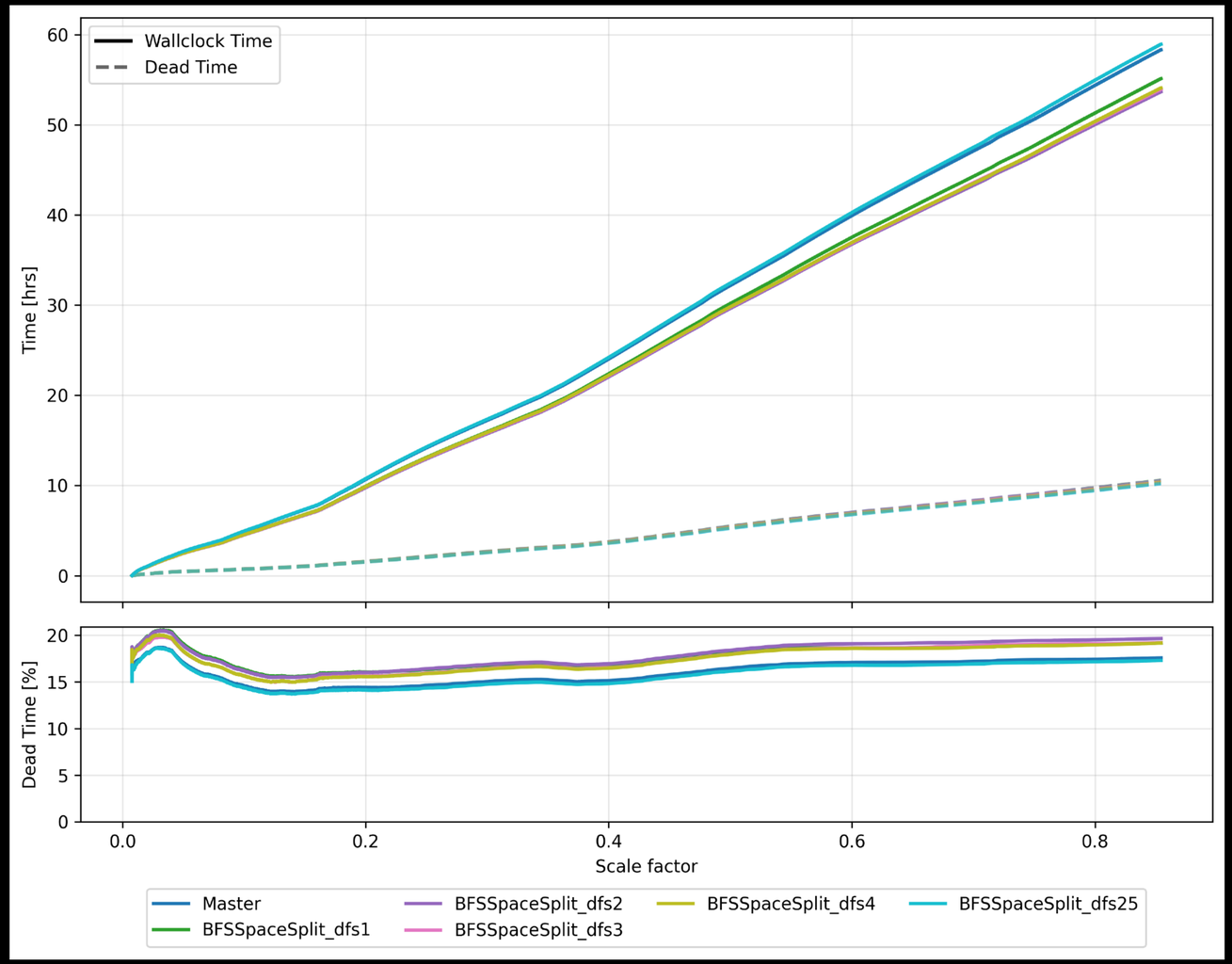
- Highly optimised DFS cell construction is great, most

space_rebuild: 26038903.1 ms / 8.4% (function execution time / percentage of total run time)

Timer Description	Time (ms)	% of Function Time	Calls	Avg/call (ms)
Function execution time	26038903.1	100.0%	17832	1460.23
├─ space_split (nested function)	21313113.3	81.9%	17832	1195.22
│ ├─ Zoom cell tree and multipole construction	20390847.9	78.3%	17832	1143.50
│ └─ Background cell tree and multipole construction	921967.0	3.5%	17832	51.70
├─ space_regrid (nested function)	1965096.8	7.5%	17833	110.19
│ ├─ space_free_cells (nested function)	703898.5	2.7%	17832	39.47
│ ├─ zoom_region_init (nested function)	630965.1	2.4%	12	52580.43
│ │ ├─ zoom_get_region_dim_and_shift (nested function)	630231.6	2.4%	17833	35.34
│ │ └─ zoom_apply_zoom_shift_to_particles (nested function...	733.6	0.0%	11	66.69
│ ├─ zoom_get_region_dim_and_shift (nested function)	630231.6	2.4%	17833	35.34
│ └─ zoom_construct_tl_cells (nested function)	1.6	0.0%	12	0.13
├─ UNACCOUNTED TIME	1400833.9	5.4%	-	-
├─ space_gparts_get_cell_index (nested function)	1352346.5	5.2%	17832	75.84
├─ hooking up cells	7491.1	0.0%	17832	0.42
└─ Moving non-local particles	21.6	0.0%	17832	0.00

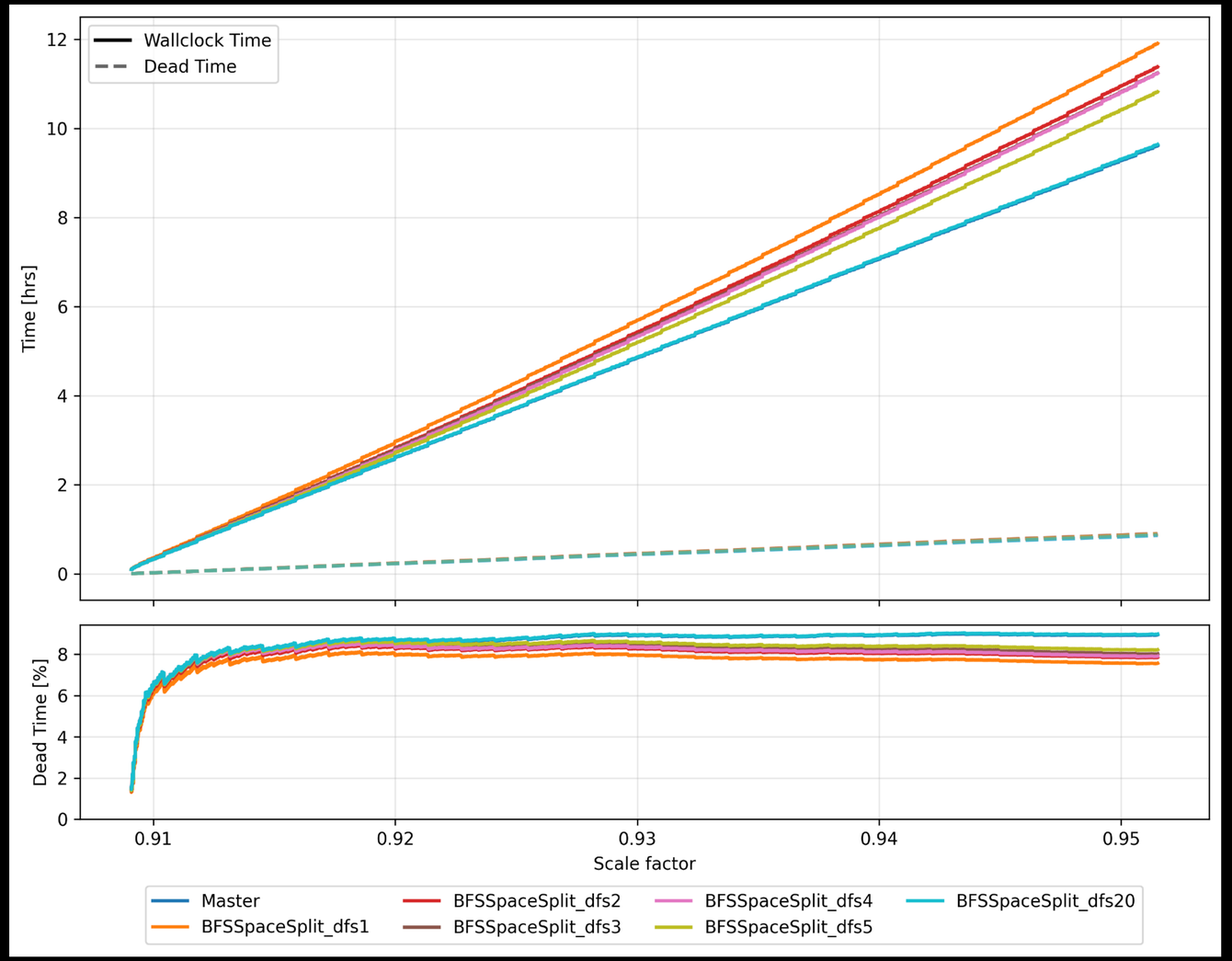
Depth First vs Breadth First

- Highly optimised DFS cell construction is great… most of the time.
- Zooms predisposed to imbalance trees leads to poor parallelisation.
- Moving to BFS can help immensely.



Depth First vs Breadth First

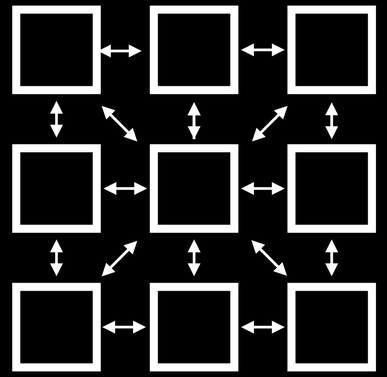
- Highly optimised DFS cell construction is great… most of the time.
- Zooms predisposed to imbalance trees leads to poor parallelisation.
- Moving to BFS can help immensely.
- BFS isn’t a golden bullet, sometimes worse, sometimes combination is needed.



The Unavoidable: Partitioning

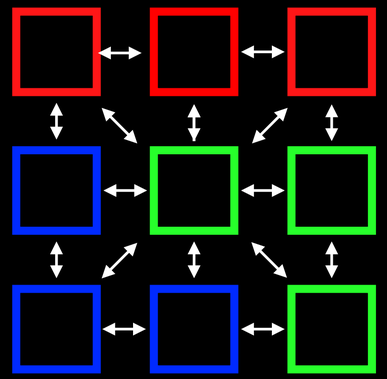
A Whistle Stop Tour: MPI

- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.



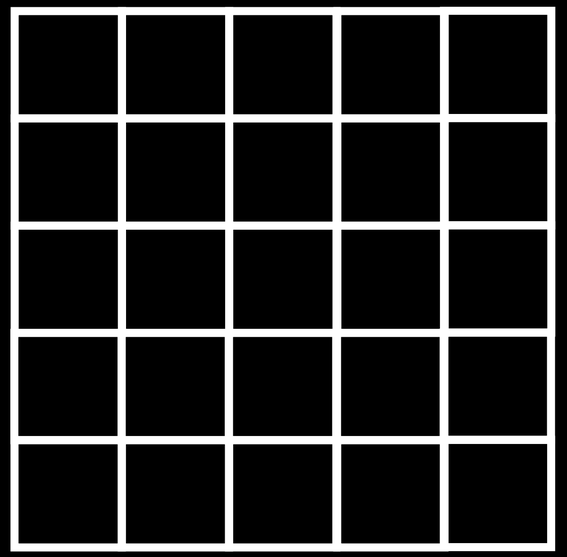
A Whistle Stop Tour: MPI

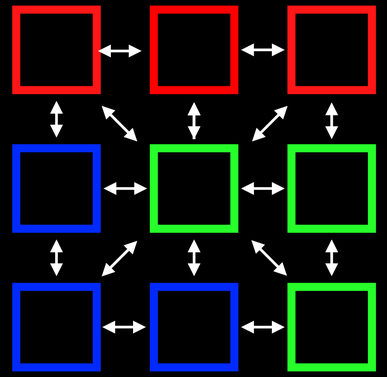
- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.



A Whistle Stop Tour: MPI

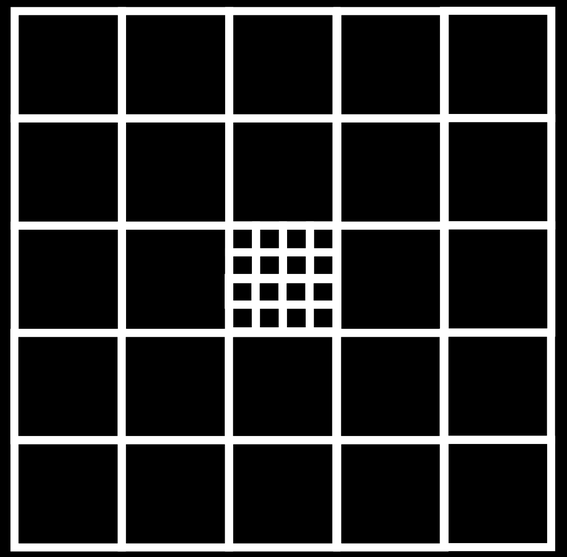
- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.

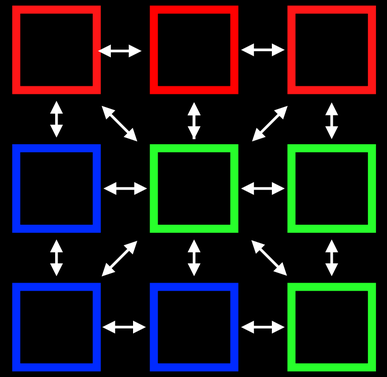




A Whistle Stop Tour: MPI

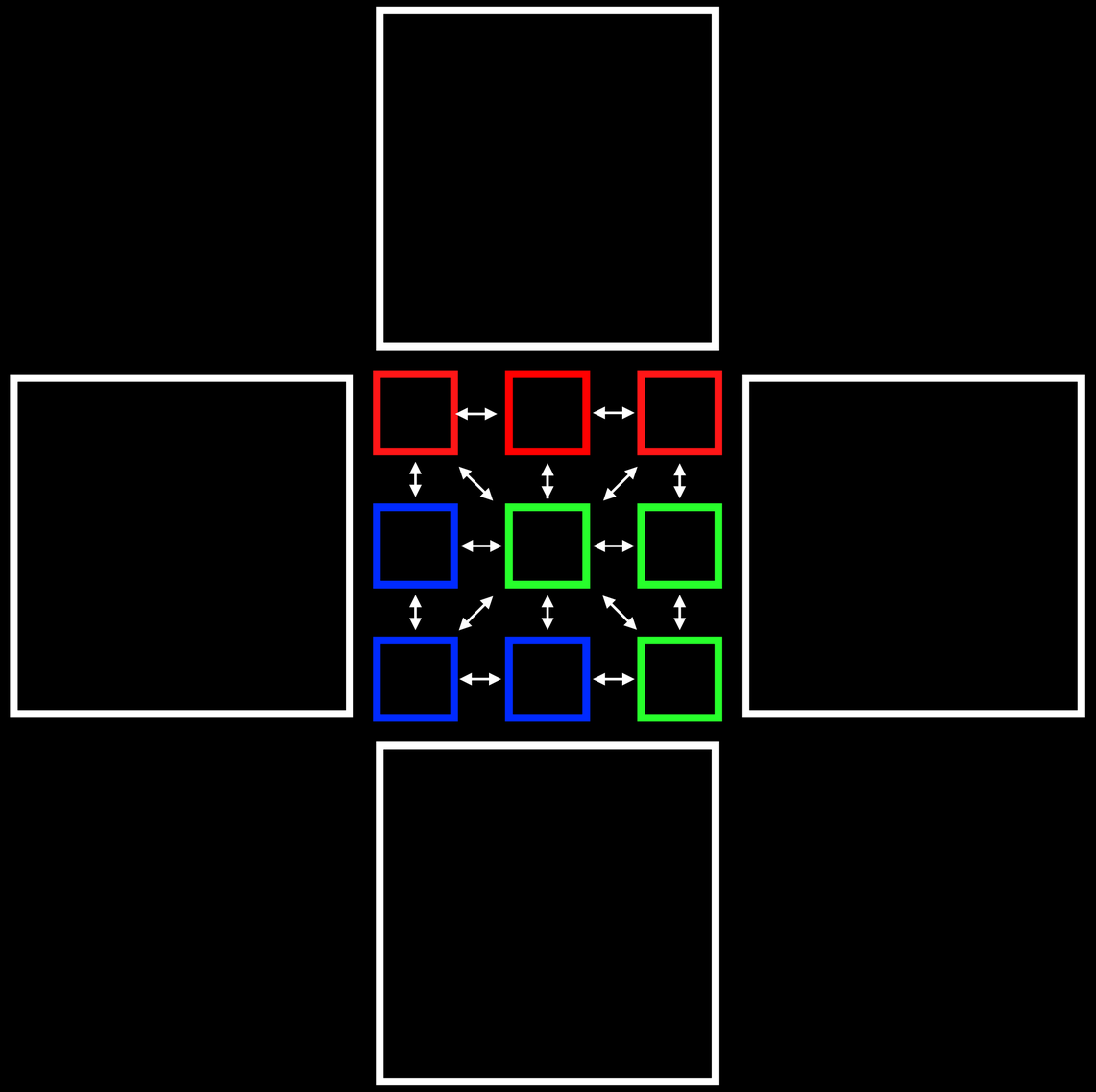
- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.





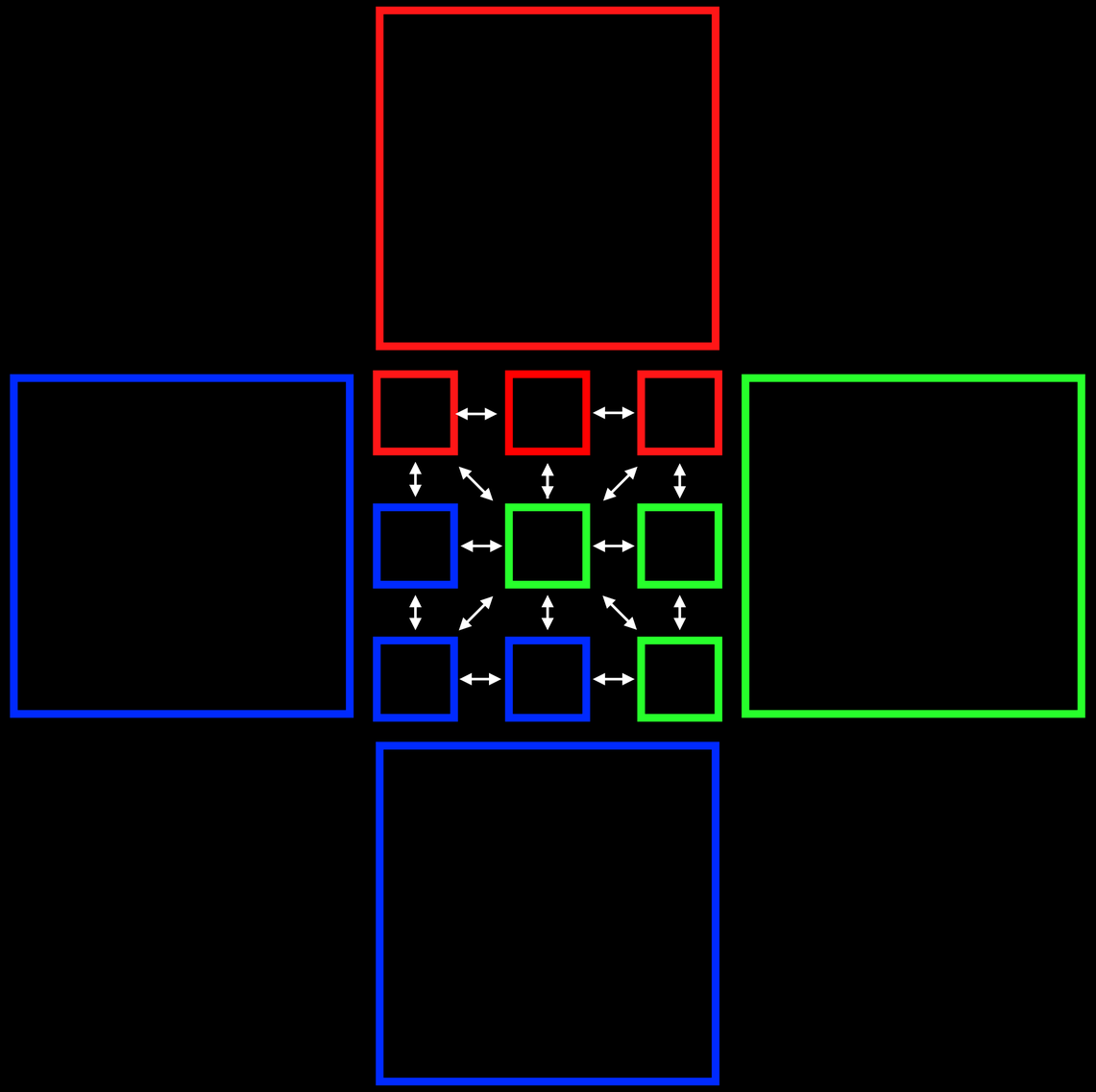
A Whistle Stop Tour: MPI

- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.



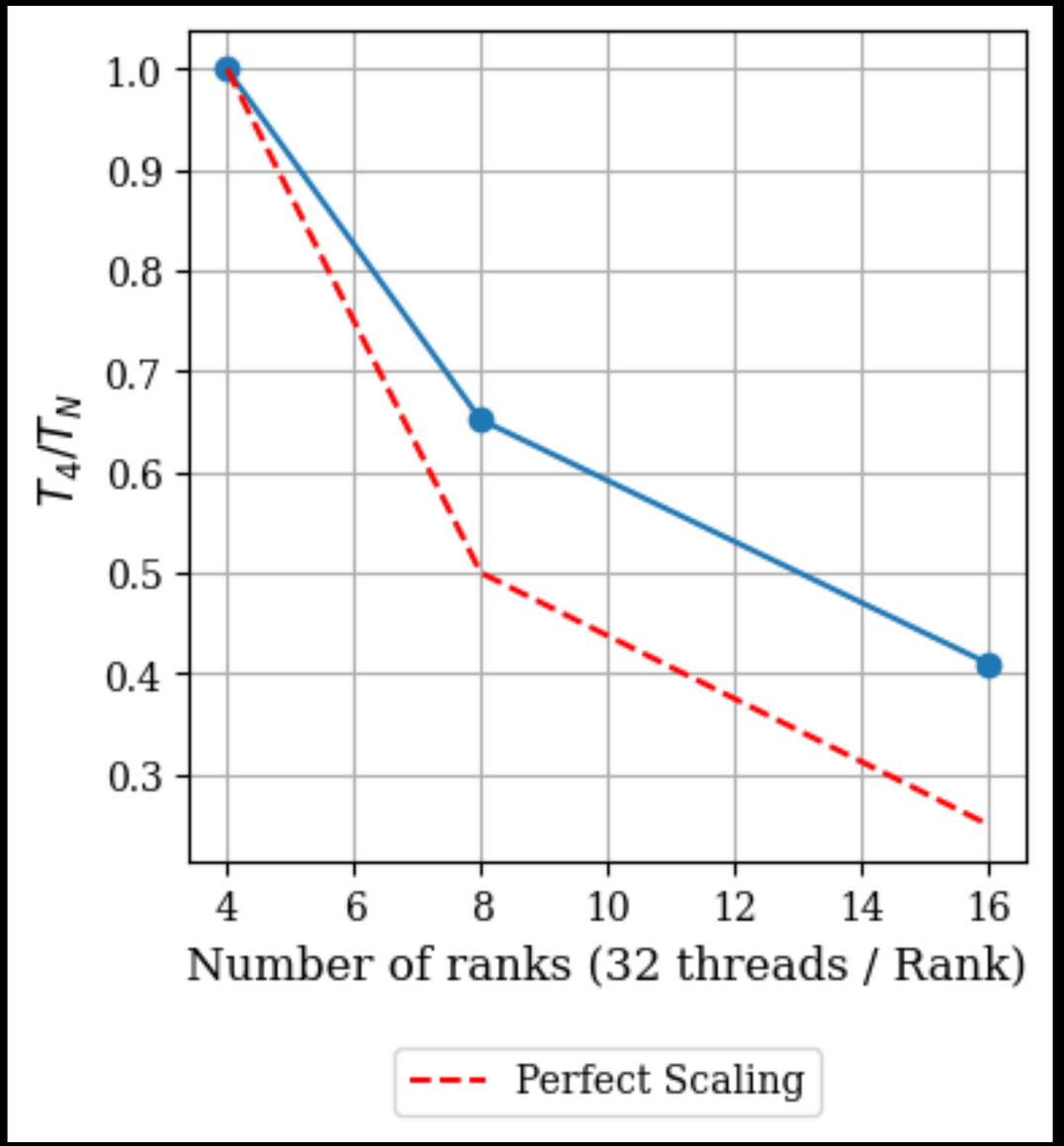
A Whistle Stop Tour: MPI

- SWIFT partitions based on work in a graph network.
- Cells are nodes with edges weighted by task cost.
- Metis partitioning used to divide the network aiming to balance work and maximise the partition volume to surface ratio.

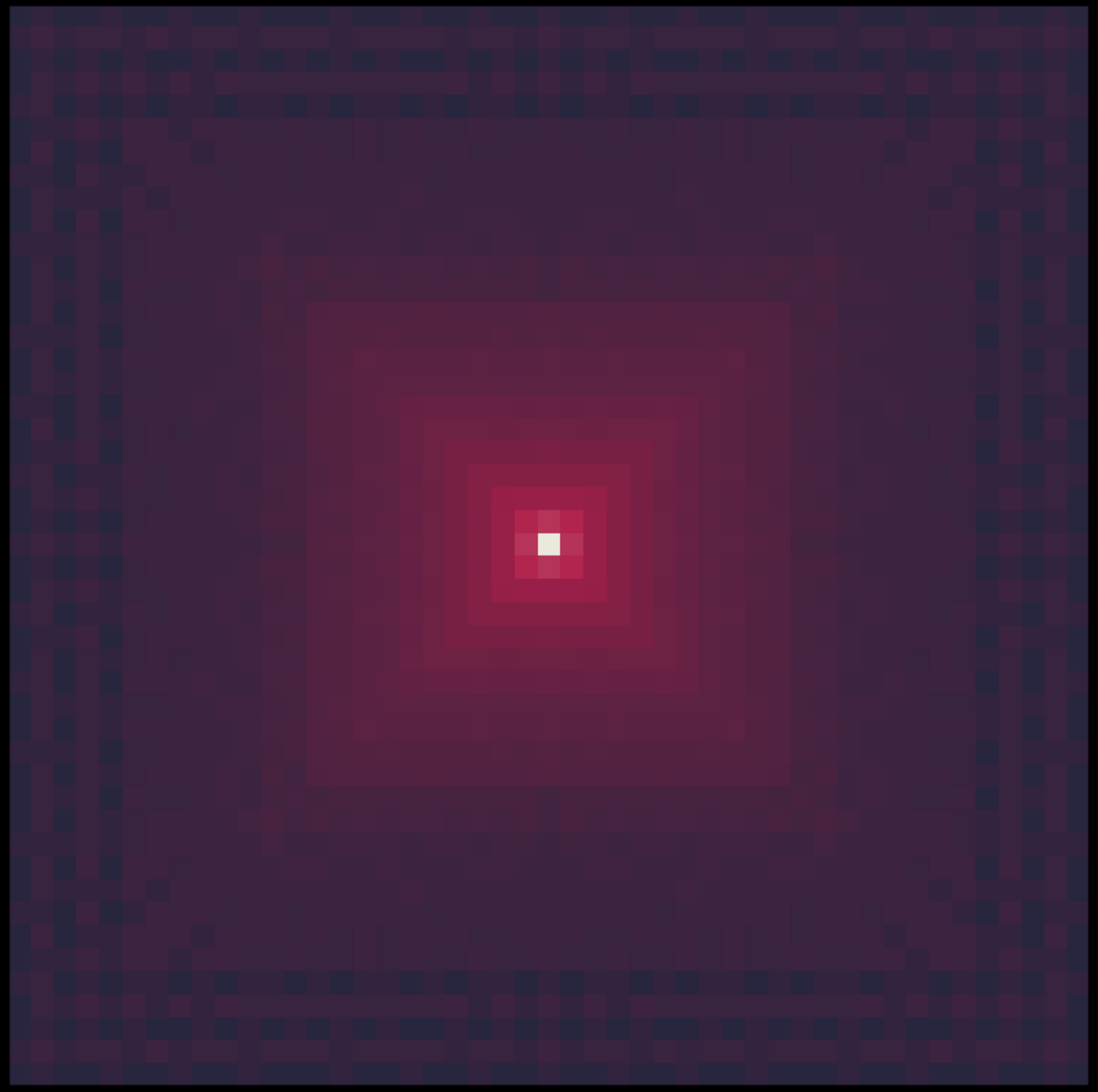


Scaling?

- Boundary exacerbates comms.
- Work is currently underway to improve the exchanges and minimise this added inefficiency.



There's A Lot Of Box Out There


5.6 Gpc

Gravity Regimes

Particle-Particle

- Short range
- Direct summation over particles.
- Expensive.
- Technically, also Particle-Multipole where possible (cheaper).

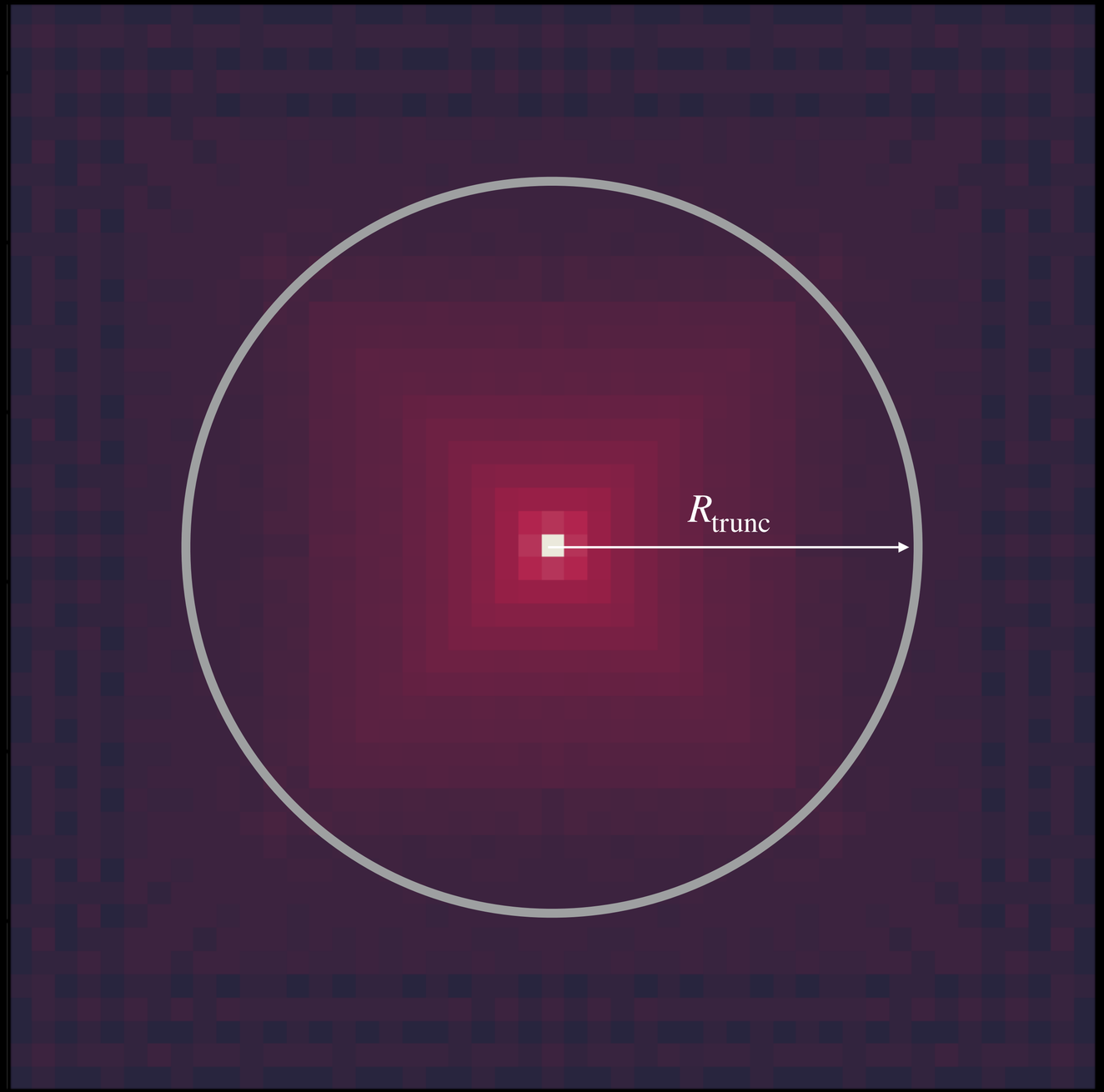
Multipole-Multipole

- Mid-long range.
- Interaction between multipoles in cells (FMM).
- Cheaper.

Particle-Mesh

- Long range (periodic) interactions.
- Cheapest.
- Higher resolution mesh -> less PP and MM.




R_{trunc}

Removing Pointless Work

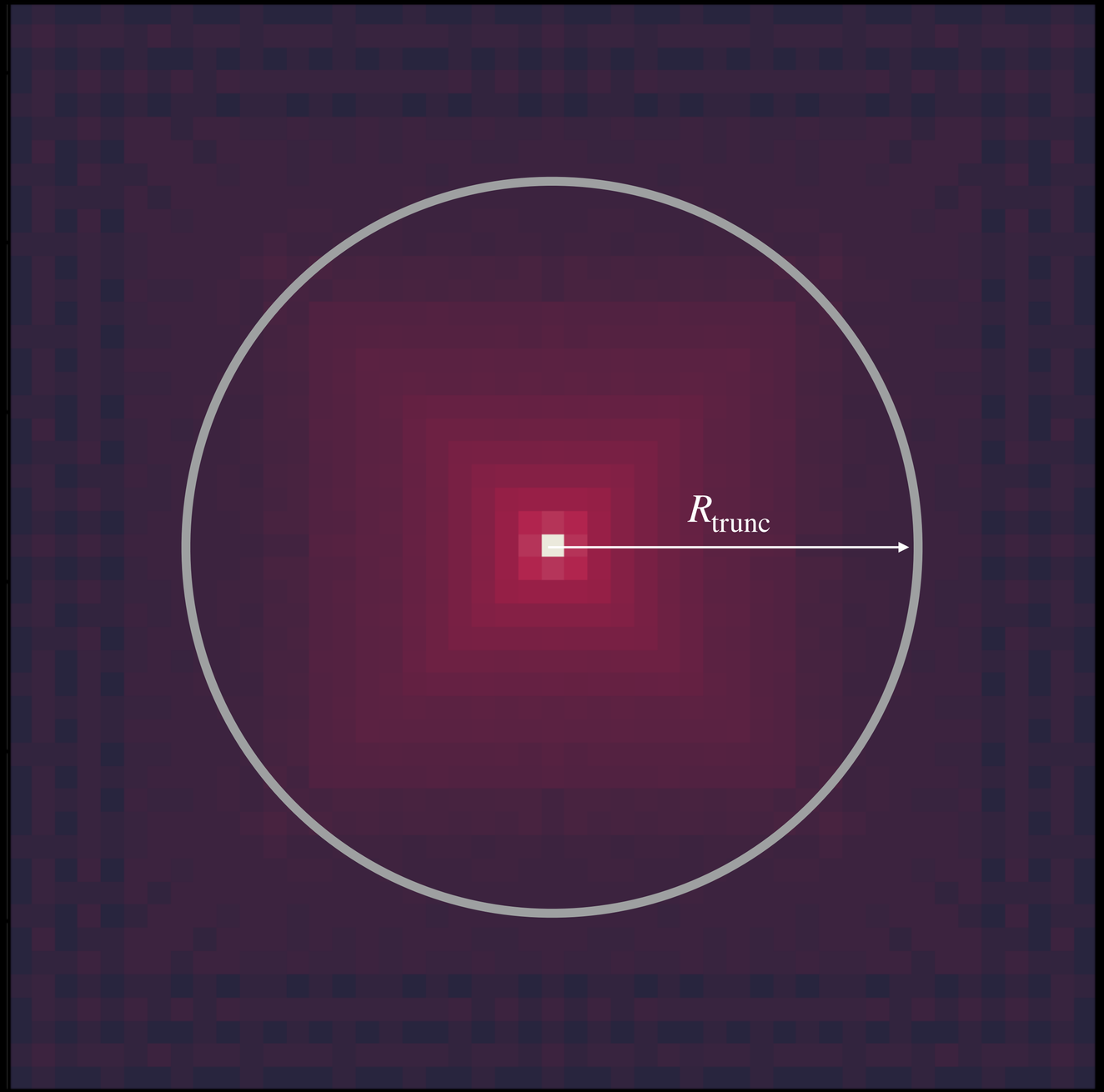
- We want the acceleration due to external mass at R to be negligible.
- Need a reference acceleration to be relative to…
- Bibbidi-Bobbidi-Boo…
- Geometric accuracy condition.

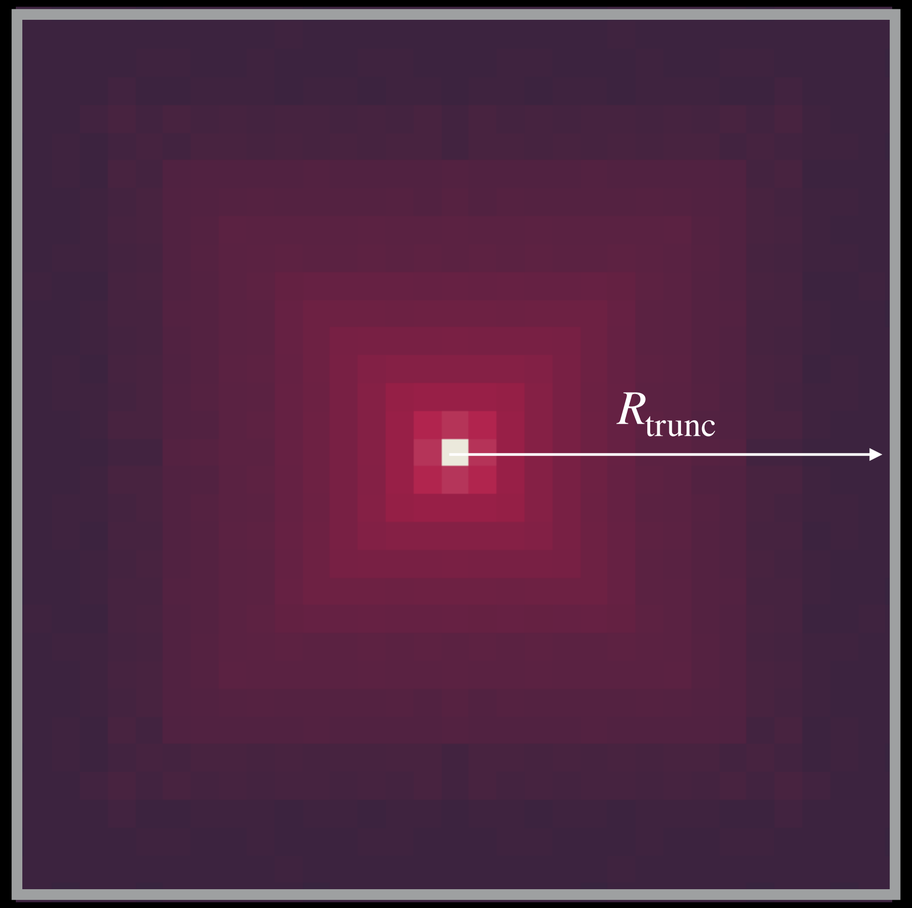
$$\|T\| \sim \frac{GM_{\text{ext}}}{R^3} \qquad a_{\text{trunc}} \leq \varepsilon \times a_{\text{ref}}$$

$$a_{\text{trunc}} \sim \|T\| \times L = \frac{GM_{\text{ext}}}{R^3} \times L$$

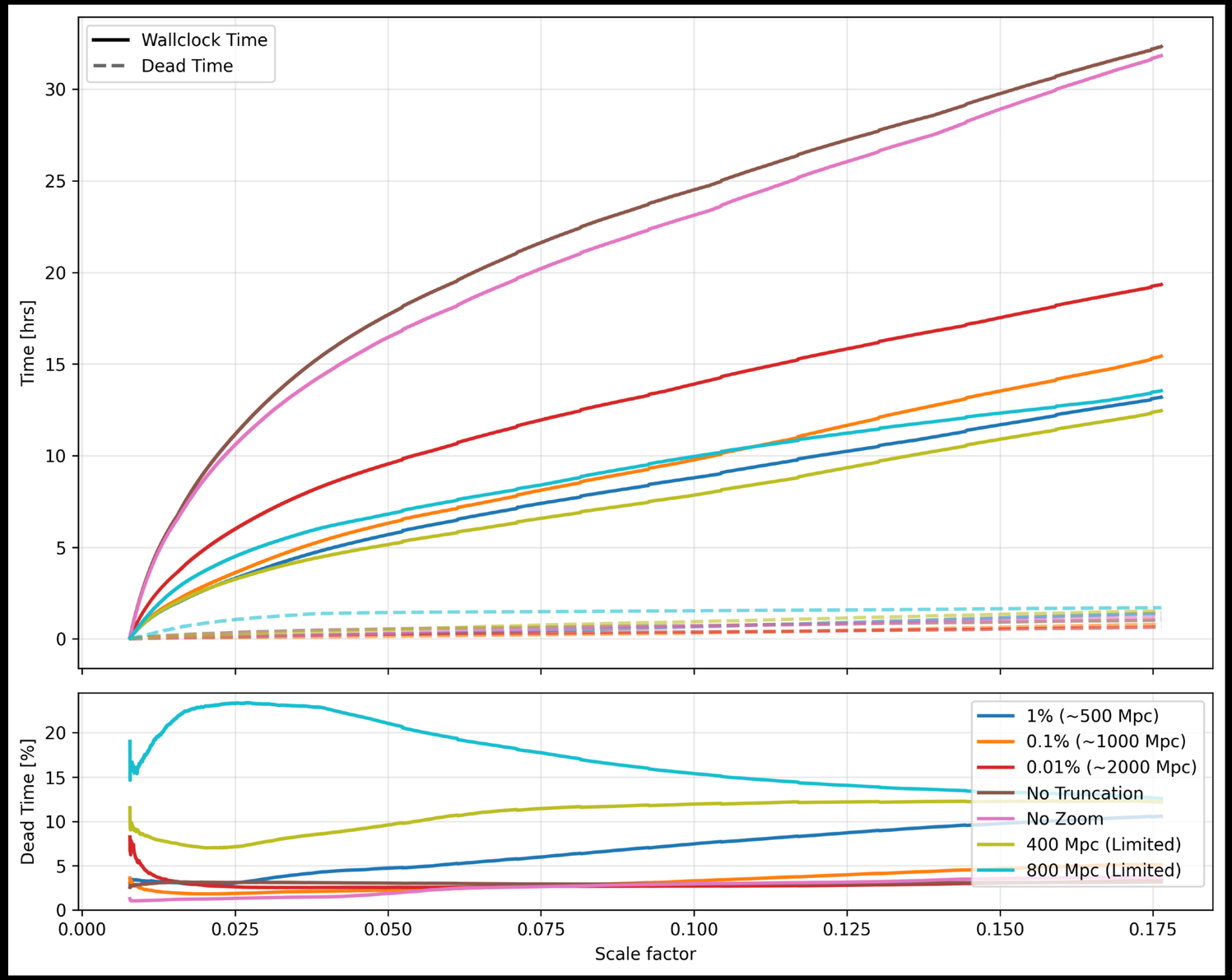
$$\frac{GM_{\text{ext}}}{R^3} \times L \leq \varepsilon \times \frac{GM_{\text{ref}}}{r^2}$$

$$R_{\text{trunc}}(\varepsilon) \geq \left(\frac{M_{\text{ext}} L^3}{\varepsilon M_{\text{zoom}} r^2} \right)^{1/3} \rightarrow R_{\text{trunc}} = \tau L \varepsilon^{-1/3}$$


R_{trunc}


R_{trunc}

Removing Background works!



Summary

- SWIFT fully capable of running optimised zoom workloads.
- Distributed zoom runs now possible with at minimum reasonable scaling.
- Zoom specific optimisations still rolling out.
- Many optimisations benefit SWIFT as a whole.
- Papers coming…

